



Portfolio

Internationale Hochschule Fernstudium
Studiengang Master of Science – Informatik

MedicalTracker: Konzeption, Softwarearchitektur und Implementierung eines digitalen Assistenzsystems für den Pflegealltag

Modul:	Projekt: Software Engineering (DLMCSPSE01_D)
Tutor:	Prof. Dr. Dirk Simon
vorgelegt von:	Steffen Niklas Oehler
Datum:	09.10.2026
Matrikelnummer:	IU14142306
E-Mail:	steffen-niklas.oehler@iu-study.org

Inhaltsverzeichnis

Inhaltsverzeichnis	II
Abbildungsverzeichnis	V
Tabellenverzeichnis	VI
Glossar	VII
1 Projektdokumentation	1
1.1 Projektübersicht.....	1
1.2 Aufbau der Arbeit	2
1.3 Abgrenzung des Systembereichs.....	3
1.3.1 Technische und architektonische Systemabgrenzung	3
1.3.2 Ergonomische Abgrenzung und Barrierefreiheit.....	3
1.3.3 Regulatorische, medizinrechtliche und datenschutzrechtliche Abgrenzung	4
1.4 Entwicklungsmethodik.....	4
1.4.1 Vorgehensmodell.....	4
1.4.2 Methodik	5
1.5 Technologie & Systemarchitektur.....	6
1.6 Risikomanagement.....	6
1.7 Zeitplanung.....	7
1.8 Benutzeranleitung	8
1.8.1 Systemvoraussetzungen	8
1.8.2 Installation und Umgebungskonfiguration.....	8
1.8.3 Datenbankinitialisierung und Testdatenbestand	9
1.8.4 Ausführung und Bedienung der Anwendung	10
1.8.5 Replikation der Testsuite und Qualitätsprüfung	10
2 Anforderungsdokumentation	12
2.1 Stakeholder-Analyse	12
2.2 Funktionale Anforderungen	13
2.3 UML-Anwendungsfalldiagramm	16
2.4 Nichtfunktionale Anforderungen	17
2.5 Datenschutz- und Sicherheitsanalyse	20

2.5.1 Einordnung nach DSGVO.....	21
2.5.2 Technische und organisatorische Schutzmaßnahmen.....	21
2.5.3 Anforderungen an einen potenziellen klinischen Realeinsatz	22
2.6 Requirements-Traceability-Matrix	22
3 Spezifikationsdokumentation	24
3.1 Objektorientiertes Datenmodell & Geschäftsobjekte.....	24
3.2 Geschäftsprozesse.....	26
3.3 Geschäftsregeln	29
3.4 Schnittstellen & Validierung.....	31
3.4.1 Systemschnittstellen	31
3.4.2 Benutzerschnittstellen & UI-Konzept	32
3.4.3 Mehrstufiges Validierungskonzept.....	34
4 Architekturdokumentation	36
4.1 Technologieübersicht	36
4.2 Architekturübersicht.....	37
4.3 Komponenten- und Klassenstruktur	39
4.4 Dynamisches Systemverhalten (Verhaltenssicht – Kernprozess GP-03)	42
5 Testdokumentation	46
5.1 Teststrategie.....	46
5.2 Testumgebung	47
5.3 Formales Testprotokoll.....	47
5.4 Analyse der Testabdeckung.....	50
5.5 Verifikation der nichtfunktionalen Anforderungen.....	50
5.5.1 Performanz- und Skalierbarkeitsbenchmark.....	50
5.5.2 Barrierefreiheit und Ergonomie	51
6 Limitationen, Risiken & Technische Schulden	52
6.1 Technische und funktionale Limitationen	52
6.2 Erfassung technischer Schulden.....	53
6.3 Retrospektive Risikobewertung.....	54
7 Fazit und Ausblick.....	55

Literaturverzeichnis.....	57
Abstract.....	58

Abbildungsverzeichnis

Abbildung 1 Anwendungsfalldiagramm.....	16
Abbildung 2 Klassendiagramm	26
Abbildung 3 Aktivitätsdiagramm.....	28
Abbildung 4 Verwaltungsdashboard	33
Abbildung 5 Patientensicht	34
Abbildung 6 Vereinfachtes Klassendiagramm des Gesamtsystems.....	40
Abbildung 7 UML-Sequenzdiagramm zur Bereitstellung der barrierefreien Tagesansicht	43
Abbildung 8 UML-Sequenzdiagramm zur Medikamenteneinnahme.....	45

Tabellenverzeichnis

Tabelle 1 Risikoanalyse	6
Tabelle 2 Zeitplanung	7
Tabelle 3 Stakeholder-Analyse	12
Tabelle 4 Requirements-Traceability-Matrix	23
Tabelle 5 Kriteriengeleitete Trade-Off-Analyse der Systemtechnologien	36
Tabelle 6 MVC Zuordnungsmatrix	38
Tabelle 7 Formales Testprotokoll der automatisierten Testsuite	48
Tabelle 8 Analyse der Testabdeckung.....	50
Tabelle 9 Erfassung technischer Schulden.....	53

Glossar

Begriff	Definition
Compliance (Medizin)	Die Zuverlässigkeit und Einhaltung der verordneten medizinischen Behandlung (z. B. pünktliche Einnahme von Medikamenten) durch die zu pflegende Person.
Flask	Ein leichtgewichtiges WSGI-Webanwendungs-Framework in Python, das nach dem Micro-Framework-Ansatz aufgebaut ist.
Jinja2	Eine weit verbreitete Template-Engine für Python, die zur dynamischen Generierung von HTML-Seiten im Flask-Backend genutzt wird.
MVP (Minimum Viable Product)	Das minimal funktionsfähige Produkt, das die Kernfunktionen (Medikation & Vitaldaten) erfüllt, um frühzeitig evaluierbar zu sein.
ORM (Object-Relational-Mapping)	Eine Technik der Softwareentwicklung, die Daten zwischen einer relationalen Datenbank (SQLite) und objektorientierten Programmiersprachen (Python) konvertiert (hier umgesetzt via SQLAlchemy).
Vitaldaten	Tägliche, messbare Körperfunktionen (z. B. Blutdruck, Puls, Körpergewicht, Blutzucker), die Aufschluss über den Gesundheitszustand liefern.
WCAG (Web Content Accessibility Guidelines)	Internationaler Standard des W3C mit Richtlinien zur Barrierefreiheit von Webinhalten. Bildet die normative Grundlage für die ergonomische Gestaltung der barrierefreien Tagesansicht (Konformitätsstufe AAA).

1 Projektdokumentation

Das erste Kapitel definiert den organisatorischen und methodischen Rahmen für die Entwicklung der Anwendung MedicalTracker. Nach der Problemabgrenzung und der Darstellung des Kundennutzens stellt die Arbeit den technologischen Rahmen vor. Anschließend werden die Projektrisiken systematisch analysiert sowie Zeitplan und Arbeitspakete strukturiert dargestellt.

1.1 Projektübersicht

Die häusliche Pflege von Angehörigen stellt für viele Menschen eine erhebliche organisatorische und emotionale Herausforderung dar. Die Koordination von Medikationsplänen, das regelmäßige Erfassen von Vitaldaten sowie die Vorbereitung auf Arzttermine erfolgen in der Praxis häufig noch auf Papier oder in unstrukturierten Notizen. Diese analogen Medienbrüche erhöhen das Risiko von Fehldosierungen, vergessenen Einnahmen und Informationsverlusten bei medizinischen Konsultationen. Da kommerzielle Pflegesysteme für den privaten Haushalt meist zu komplex sind und gängige Cloud-Apps bei sensiblen Gesundheitsdaten datenschutzrechtliche Vorbehalte wecken, besteht ein klarer Bedarf an einer fokussierten, lokalen Lösung.

Das Projekt adressiert diese Problematik durch die Konzeption und Implementierung einer intuitiven, lokalen Webanwendung. MedicalTracker dient als zentrales digitales Assistenzsystem für den Pflegealltag. Die Anwendung verfolgt das Ziel, pflegende Angehörige bei der täglichen Dokumentation und Planung zu entlasten, der behandelnden Ärzteschaft valide Verlaufsdaten bereitzustellen und zu pflegenden Personen durch eine barrierefreie Tagesansicht mehr Selbstständigkeit zu ermöglichen. Aus dieser Zielsetzung leitet sich die zentrale Projekt- und Forschungsfrage der Arbeit ab:

„Wie muss eine lokale, leichtgewichtige Webanwendung architektonisch konzipiert sein, um pflegende Angehörige bei der Medikations- und Vitaldatenverwaltung zu entlasten? Wie lässt sich dabei eine barrierefreie Nutzung für pflegebedürftige Personen realisieren, ohne die rechtliche Komplexität eines zertifizierten Medizinprodukts zu erzeugen?“

Der vollständige Quellcode der Webanwendung, sämtliche Konfigurationsdateien, automatisierte Testskripte sowie die begleitenden Dokumente werden versioniert über das folgende GitHub-Repository bereitgestellt:

<https://github.com/SteffenOe/medicaltracker.git>

1.2 Aufbau der Arbeit

Die vorliegende Portfolioarbeit dokumentiert den vollständigen Lebenszyklus des Entwicklungsvorhabens und gliedert sich entlang der drei Phasen des Software-Engineering-Prozesses in sieben aufeinander aufbauende Hauptkapitel.

Den Auftakt bildet das vorliegende Kapitel 1 (Projektdokumentation), welches den organisatorischen und methodischen Rahmen absteckt, die gesellschaftliche Problemstellung motiviert und die zentrale Forschungsfrage formuliert. Zudem wird der Systembereich in technischer, ergonomischer und medizinrechtlicher Hinsicht dreidimensional abgegrenzt, das phasenorientierte Vorgehensmodell begründet sowie das präventive Risikomanagement samt Meilensteinplanung etabliert. Daran anknüpfend formalisiert Kapitel 2 (Anforderungsdokumentation) die Anforderungen aus Sicht der identifizierten Stakeholder. Die fachlichen Vorgaben werden anhand von sieben priorisierten IREB-User-Stories (FA-01 bis FA-07) samt Akzeptanzkriterien operationalisiert und in einem UML-Anwendungsfalldiagramm visualisiert. Ergänzend definiert das Kapitel die Qualitätsanforderungen (NFA-01 bis NFA-07) gemäß ISO/IEC 25010. Ergänzt um eine detaillierte Datenschutz- und Sicherheitsanalyse (DSGVO, MDR) gewährleistet eine abschließende Traceability-Matrix die lückenlose Verfolgbarkeit aller Anforderungen über den gesamten Entwicklungsverlauf hinweg.

Im darauffolgenden Entwurfsschritt widmet sich Kapitel 3 (Spezifikationsdokumentation) der fachlichen Modellierung. Hierbei wird das statische, objektorientierte Datenmodell samt Multiplizitäten hergeleitet. Der Ablauf der Kernprozesse, insbesondere der Einnahme- und Bestandsführung (GP-03), wird anhand eines UML-Aktivitätsdiagramm detailliert dargestellt. Abschließend werden die Geschäftsregeln (GR-01 bis GR-04) fixiert. Abgerundet wird die Spezifikation durch die Definition der Schnittstellen, das duale Benutzerschnittstellenkonzept sowie ein mehrstufiges Validierungsmodell. Die softwaretechnische Realisierung wird in Kapitel 4 (Architekturdokumentation) offengelegt. Ausgehend von einer kriteriengeleiteten Trade-Off-Matrix der Systemtechnologien wird die hierarchische Drei-Schichten-Architektur nach dem MVC- und Service-Layer-Pattern dokumentiert, die modulare Komponenten- und Klassenstruktur analysiert und das dynamische Laufzeitverhalten des Kernprozesses anhand zweier detaillierter UML-Sequenzdiagramme abgebildet.

Den Schwerpunkt der Qualitätssicherung in Phase 3 markiert Kapitel 5 (Testdokumentation). Entlang der Testpyramide und des Standards ISO/IEC/IEEE 29119 wird die automatisierte Testsuite dokumentiert und ein 17-teiliges formales Testprotokoll vorgelegt. Zudem verifizieren empirische Lasttests mit 10.000 Datensätzen sowie typografische Inspektionen die Einhaltung der nichtfunktionalen Anforderungen an Performanz, Skalierbarkeit und Barrierefreiheit. Im Anschluss unterzieht Kapitel 6 (Limitationen, Risiken & Technische Schulden) das MVP einer kritischen Reflexion. Neben den funktionalen und architektonischen Systemgrenzen werden bewusst eingegangene technische Schulden (TD-01 bis TD-04) transparent katalogisiert und mit konkreten Refactoring-Pfaden versehen, bevor ein retrospektiver Abgleich die Wirksamkeit der anfänglichen Risikoanalyse evaluiert. Den Abschluss bildet Kapitel 7 (Fazit und Ausblick), welches die Projektergebnisse thesenartig

synthetisiert, die zentrale Forschungsfrage anhand der drei Gestaltungsdimensionen Architektur, Ergonomie und Rechtssicherheit beantwortet und strategische Entwicklungsperspektiven für den künftigen Realeinsatz aufzeigt.

1.3 Abgrenzung des Systembereichs

Die Definition der Systemgrenzen ist für sicherheitskritische Informationssysteme im Gesundheits- und Pflegekontext von zentraler Bedeutung. Um Fehlinterpretationen bezüglich des Einsatzzwecks, der Haftungsrisiken und der technischen Betriebsbedingungen auszuschließen, erfolgt die Systemabgrenzung entlang dreier Dimensionen: (1) Technische Architektur und Infrastruktur, (2) Ergonomie und Barrierefreiheit (Human-Machine Interaction) sowie (3) Regulierung, Medizinrecht und Datenschutz.

1.3.1 Technische und architektonische Systemabgrenzung

Auf architektonischer Ebene fokussiert das System auf maximale Autarkie und minimale Systemvoraussetzungen beim Endanwendenden:

In-Scope: Lokaler Standalone-Betrieb als zustandsbehaftete Webanwendung unter Microsoft Windows 10/11. Ausführung über einen lokalen WSGI-Server unter Nutzung einer Dateidatenbank. Die Persistenzschicht kapselt relationale Integritätsprüfungen (Transaktionssicherheit) atomar ab. Der PDF-Export operiert flüchtig im Arbeitsspeicher, um ein persistentes Zwischenspeichern unverschlüsselter Berichte im temporären Dateisystem zu unterbinden.

Out-of-Scope: Verteilte Cloud-Infrastrukturen, Hosting auf externen Webservern sowie REST-Schnittstellen zur Anbindung externer Drittsysteme oder nativer Mobile-Apps. Ebenso explizit ausgeschlossen ist ein Multi-Tenancy-Betrieb (Mandantenfähigkeit): Die Anwendung implementiert im aktuellen Reifegrad keine softwarebasierte Mehrbenutzerverwaltung mit dynamischer Rollen- und Rechteverwaltung auf Betriebssystem- oder Datenbankebene.

1.3.2 Ergonomische Abgrenzung und Barrierefreiheit

Hinsichtlich der Mensch-Maschine-Schnittstelle differenziert das System strikt nach dem kognitiven und sensorischen Profil der agierenden Rollen:

In-Scope: Bereitstellung eines dualen Benutzerschnittstellenkonzepts. Für pflegende Angehörige steht ein funktionsdichtes Verwaltungsdashboard bereit. Für die zu pflegende Person wird ein isolierter, reduzierter Patientenmodus (Tagesansicht) realisiert. Dieser setzt Kernkriterien der Web Content Accessibility Guidelines (WCAG) um. Großflächige Interaktionselemente, eine

Grundschriftgröße von mindestens 18 pt sowie Kontrastverhältnisse von mindestens 7:1 bei handlungsleitenden Schaltflächen.

Out-of-Scope: Spezialisierte Hilfsmittelunterstützungen für Personen mit vollständiger Erblindung sowie dynamische Text-to-Speech-Engines oder responsive Touch-Gestensteuerungen. Das System adressiert im MVP spezifisch visuelle und feinmotorische Alterseinschränkungen, nicht jedoch komplexe multimediale Barrierefreiheitsanforderungen.

1.3.3 Regulatorische, medizinrechtliche und datenschutzrechtliche Abgrenzung

Die Abgrenzung des Rechtsrahmens determiniert die Klassifikation des Gesamtsystems:

In-Scope: Einsatz als reines Dokumentations-, Orientierungs- und Bestandsassistenzsystem im privaten häuslichen Umfeld. Das System dient ausschließlich als digitales Nachschlagewerk und Kommunikationsmedium zwischen Haushalt und Hausarztpraxis (via strukturiertem PDF-Export).

Out-of-Scope: Das System stellt explizit kein Medizinprodukt dar. Ausgeschlossen sind automatische Dosierungsberechnungen, Wechselwirkungsanalysen, Vitalwert-Interpretationen oder automatisierte diagnostische Warnmeldungen. Das System trifft keine therapeutischen Entscheidungen. Die medizinische Bewertung obliegt ausnahmslos dem behandelnden Fachpersonal. Ebenso ist der kommerzielle Einsatz in ambulanten oder stationären Pflegebetrieben ausgeschlossen.

1.4 Entwicklungsmethodik

Um eine hohe Softwarequalität und Termintreue im Rahmen des Projekts zu gewährleisten, basiert die Entwicklung auf einem strukturierten Methodenset. Nachfolgend werden das angewendete Vorgehensmodell sowie die Methodik zur Anforderungs- und Spezifikationsanalyse systematisch zusammengeführt.

1.4.1 Vorgehensmodell

Die Wahl des Prozessmodells unterliegt im Kontext sicherheitskritischer Gesundheitssoftware einem klassischen Dilemma des Software Engineerings: Während agile Vorgehensmodelle (z. B. Scrum) maximale Flexibilität bei unklaren Anforderungen bieten, erfordert der Umgang mit Medikations- und Vitaldaten eine frühzeitige, lückenlose Spezifikation von Geschäftsregeln (GR-01 bis GR-04) und Sicherheitsgrenzen vor der ersten Codezeile.

Aus diesem Grund folgt das Vorhaben einem phasenorientierten, sequenziellen Entwicklungsprozess in Anlehnung an das V-Modell, modifiziert durch iterative Prototyping-Schleifen an der Benutzerschnittstelle:

1. Rationale für die Sequenzierung: Die formale Trennung von Konzeption (Phase 1), Architektur/Implementierung (Phase 2) und Qualitätssicherung (Phase 3) minimiert fundamentale Entwurfsrisiken. Durch die vollständige Modellierung des relationalen Schemas und der Kontrollflüsse (UML-Aktivitätsdiagramm) vor der Backend-Implementierung wurden kostspielige Architektur-Refactorings vermieden.
2. Abgrenzung gegenüber reinen agilen Modellen: Ein rein iterativ-inkrementelles Vorgehen (z. B. Scrum-Sprints ohne Vorab-Spezifikation) hätte im Rahmen einer individuellen Prüfungsleistung mit festen Meilensteinen einen unverhältnismäßigen organisatorischen Overhead erzeugt. Zudem birgt agile Architecturevolution bei persistenzkritischen Systemen das Risiko inkonsistenter Datenbankmigrationen.
3. Mitigation des sequenziellen Risikos: Die inhärente Schwachstelle des linearen Wasserfallansatzes, das späte Feedback zu Usability und Ergonomie, wurde methodisch durch frühzeitige Erstellung kognitiv reduzierter UI-Mockups kompensiert (vgl. Abbildung 4 und 5). Dies erlaubte die Verifikation der Barrierefreiheitskriterien (WCAG 2.2), bevor die Templates implementiert wurden.

1.4.2 Methodik

Zur systematischen Erarbeitung der Artefakte werden etablierte Standardmethoden des Software Engineerings miteinander kombiniert. Die Erhebung der funktionalen Anforderungen erfolgt aus Sicht der identifizierten Akteure in Form agiler User Stories, die gemäß dem IREB-Standard strukturiert werden. Um eine fokussierte Softwareentwicklung zu garantieren und Terminrisiken zu minimieren, werden alle Anforderungen im Hinblick auf das Minimum Viable Product (MVP) in die Prioritätsstufen Hoch, Mittel und Niedrig eingestuft. Als Anforderungen hoher Priorität gelten dabei die unverzichtbaren Kernfunktionen für den primären Systemzweck. Hierunter fallen die Medikationsverwaltung, die Vitaldatenerfassung, eine barrierefreie Tagesansicht sowie grundlegende Sicherheitsfunktionen (FA-01, FA-02, FA-05, FA-06, FA-07 sowie NFA-01 bis NFA-07). Anforderungen mittlerer Priorität umfassen wichtige Zusatzfunktionen für den erweiterten Praxiseinsatz, wie etwa die Generierung eines PDF-Arztberichts oder die Terminverwaltung (FA-03, FA-04). Anforderungen mit niedriger Priorität werden für den Rahmen des aktuellen Entwicklungsvorhabens zurückgestellt und als Out-of-Scope eingeordnet.

Die visuelle System- und Prozessmodellierung erfolgt mittels Anwendungsfall-, Klassen- und Aktivitätsdiagrammen mit Swimlanes. Das Qualitäts- und Risikomanagement stützt sich hinsichtlich der Qualitätskriterien auf das internationale Standardmodell ISO/IEC 25010:2023. Projektrisiken werden

quantitativ über eine zweidimensionale Risikomatrix aus dem Produkt von Eintrittswahrscheinlichkeit und Auswirkung evaluiert und mit entsprechenden Präventivmaßnahmen versehen.

1.5 Technologie & Systemarchitektur

Die Festlegung der technischen Systemarchitektur und der konkreten Basistechnologien erfolgt nachgelagert in Kapitel 4 auf Basis der in Kapitel 2 und 3 ermittelten Qualitätsanforderungen und Schnittstellenspezifikationen.

1.6 Risikomanagement

Zur frühzeitigen Identifikation und Reduktion von Projektrisiken wird eine systematische Risikoanalyse durchgeführt. Die Risiken werden nach Eintrittswahrscheinlichkeit und Auswirkung bewertet und mit konkreten Gegenmaßnahmen (Mitigation) versehen.

Tabelle 1 Risikoanalyse

Risiko-ID	Beschreibung	Eintrittswahrscheinlichkeit	Auswirkung	Gegenmaßnahme
R-01 (Technisch)	Datenverlust / -korruption Plötzlicher Serverabsturz beschädigt die dateibasierte Datenbank.	Gering	Hoch	Einsatz eines sicheren Transaktionsmanagements; automatisierte Erstellung eines Backups der .db-Datei bei jedem App-Start.
R-02 (Design)	Mangelnde Usability (Barrierefreiheit) Die ältere Zielgruppe (Patienten) scheitert an zu kleinen Klickelementen oder komplexer Menüführung.	Mittel	Hoch	Konsequenter „Mobile-First“- und High-Contrast-Ansatz; Erstellung von UI-Prototypen vor der finalen Programmierung.
R-03 (Ressourcen)	Verzögerungen durch externe Faktoren Berufliche Spitzenbelastungen blockieren die Weiterentwicklung innerhalb des Zeitplans.	Mittel	Mittel	Definition eines Minimal Viable Products (MVP). Kernfunktionen (Medikation & Vitalwerte) besitzen höchste Priorität; optionale Features (PDF-Export) dienen als Puffer.

Quelle: eigene Darstellung

Die Auswertung der Risikomatrix zeigt, dass insbesondere das Design-Risiko (R-02) eine hohe Auswirkung auf den Gesamterfolg und den Kundennutzen der Anwendung hat. Durch die frühzeitige Berücksichtigung von Richtlinien zur Barrierefreiheit wird diesem Risiko bereits im Entwurf gezielt entgegengewirkt.

1.7 Zeitplanung

Die zeitliche Strukturierung des Projekts orientiert sich an einer veranschlagten Netto-Projektlaufzeit von 8 Wochen. Der Zeitplan verteilt die wesentlichen Arbeitspakete auf die drei vorgegebenen Phasen des Portfoliokurses.

Tabelle 2 stellt die Zeitplanung, die zugehörigen Arbeitspakete sowie die zentralen Qualitäts-Meilensteine (Einreichungen) dar.

Tabelle 2 Zeitplanung

Phase	Zeitraum	Wesentliche Arbeitspakete & Meilensteine
Phase 1: Konzeption	Woche 1-2	• Erstellung der Projektdokumentation (Einleitung, Risikomanagement, Zeitplan) • Erstellung des Anforderungsdokuments (Stakeholder, User Stories, NFAs, Glossar) • Erstellung des Spezifikationsdokuments (Datenmodell, Geschäftsprozesse, Schnittstellen) • Modeling der UML-Diagramme (Use-Case, Klasse, Aktivität) • Meilenstein 1: Abgabe Portfolioteil 1 & Feedback-Schleife
Phase 2: Architektur & Implementierung	Woche 3-6	• Erstellung des Architekturdokuments (Technologieentscheidung, UML-Sequenzdiagramm) • Aufsetzen des GitHub-Repositories und der Ordnerstruktur • Backend-Implementierung (Flask, Datenmodelle, Routing) • Frontend-Implementierung (Templates und Layouts) • Befüllen der Datenbank mit Testdaten • Meilenstein 2: Abgabe Portfolioteil 2 & Feedback-Schleife
Phase 3: Finalisierung	Woche 7-8	• Erstellung des Testdokuments (Teststrategie, Testprotokolle) • Schreiben der Benutzeranleitung & Erfassen technischer Schulden

		• Verfassen des 2-seitigen Abstracts („Making-of“ & Reflexion) • Zusammenfügen der Dokumente & ZIP-Export für GitHub • Meilenstein 3: Finale Einreichung der Prüfungsleistung
--	--	---

Quelle: eigene Darstellung

1.8 Benutzeranleitung

Die nachfolgende Benutzer- und Installationsanleitung dokumentiert die technischen Systemvoraussetzungen, das softwaretechnische Deployment sowie die praktische Bedienung. Ziel ist es, eine barrierefreie, installationsleichte Inbetriebnahme (Zero-Configuration-Deployment) im lokalen Nutzungskontext zu ermöglichen.

1.8.1 Systemvoraussetzungen

Die Anwendung ist als lokale Desktop-Webanwendung konzipiert und setzt auf dem Zielsystem ausschließlich Standardkomponenten voraus:

- **Betriebssystem:** Microsoft Windows 10 oder Windows 11 (64-Bit-Architektur).
- **Laufzeitumgebung:** Python-Interpreter ab Version 3.14 (empfohlen und getestet: Python 3.14+).
- **Paketmanager:** pip (standardmäßig in Python-Distributionen integriert).
- **Webbrowser:** Jeder moderne, standardkonforme Webbrowser (z. B. Microsoft Edge, Google Chrome, Mozilla Firefox) mit aktiviertem JavaScript.
- **Hardware-Ressourcen:** Mindestens 50 MB freier Festplattenspeicher sowie 512 MB verfügbarer Arbeitsspeicher (RAM).

1.8.2 Installation und Umgebungskonfiguration

Die Auslieferung und Bereitstellung erfolgen quelloffen über das offizielle GitHub-Repository. Die Einrichtung gliedert sich in folgende Schritte:

1. Repository klonen oder ZIP entpacken:

Öffnen Sie eine Eingabeaufforderung (PowerShell oder Windows Terminal) und klonen Sie das Projektverzeichnis.

PowerShell

```
git clone https://github.com/SteffenOe/medicaltracker.git
cd medicaltracker
```

2. Isolierte virtuelle Umgebung erstellen und aktivieren:

Um Versionskonflikte mit systemweiten Python-Paketen auszuschließen, wird die Verwendung eines Virtual Environments (venv) empfohlen.

```
PowerShell

python -m venv venv
.\venv\Scripts\Activate.ps1
```

(Hinweis: Bei Verwendung der Standard-Eingabeaufforderung cmd.exe lautet der Aktivierungsbefehl `venv\Scripts\activate.bat`.)

3. Abhängigkeiten installieren:

Installieren Sie die im Projektstammverzeichnis hinterlegten Third-Party-Bibliotheken (u. a. Flask, Flask-SQLAlchemy, Flask-WTF, ReportLab) über den Paketmanager.

```
PowerShell

pip install -r requirements.txt
```

4. Umgebungsvariablen konfigurieren (optional):

Im Stammverzeichnis befindet sich eine Datei `.env.example`. Für den lokalen Entwicklungs- und Evaluierungsbetrieb greift das System auf vordefinierte, sichere Fallback-Werte (u. a. generierter CSRF-Secret-Key und lokale SQLite-URI) zurück. Für individuelle Konfigurationen kann die Datei als `.env` kopiert und angepasst werden:

```
PowerShell

copy .env.example .env
```

1.8.3 Datenbankinitialisierung und Testdatenbestand

Um Erstnutzern eine sofortige Evaluation ohne manuellen Vorerfassungsaufwand zu ermöglichen, enthält das Projekt ein dediziertes Seeding-Skript.

```
PowerShell

python seed.py
```

Das Skript initialisiert das relationale Datenbankschema in der Datei `medical_tracker.db` und befüllt das System mit einem konsistenten, realistischen Testdatensatz für die Musterpatientin Anna Muster:

- Vollständige Patientenstammdaten inklusive Notfallkontakt.
- Drei verordnete Arzneimittel (Ramipril 5mg, Ibuprofen 400mg, Vitamin D3) mit definierten Tageszeiten, Sollbeständen und Mindestbestandswerten.
- Historisierte Vitaldatenverläufe (Blutdruck, Puls, Körpergewicht) über mehrere Tage.
- Anstehende und erledigte Pflegetermine im Alltagsplaner.

1.8.4 Ausführung und Bedienung der Anwendung

Nach erfolgreicher Installation wird der integrierte lokale Webserver über das Hauptskript gestartet.

```
PowerShell  
  
python run.py
```

Die Konsole quittiert den Start mit dem Statushinweis, dass der Server auf dem lokalen Host-Interface lauscht. Öffnen Sie anschließend Ihren Webbrowser und navigieren Sie zur Adresse <http://127.0.0.1:5000/>

Die Anwendung implementiert ein duales Benutzerschnittstellenkonzept, das über zwei Hauptansichten bedient wird.

1. Verwaltungsdashboard für pflegende Angehörige:

- **Übersicht:** Das zentrale Dashboard bietet schnellen Zugriff auf aktuelle Vitalwerte, Terminübersichten und akute Warnhinweise.
- **Medikationsverwaltung (/medikamente/):** Ermöglicht das Anlegen, Editieren und Löschen von Medikamenten. Sobald ein Bestand den definierten Mindestschwellenwert erreicht oder unterschreitet, wird automatisch eine rote Warnung (Achtung: Nachbestellung erforderlich!) eingeblendet.
- **Vitaldaten-Tracking (/vitaldaten/):** Dient der strukturierten Erfassung von Blutdruck, Puls und Gewicht mit automatischer Plausibilitätsprüfung (GR-02) und tabellarischer Verlaufshistorie.
- **Terminplaner (/termine/):** Pflege- und Arzttermine können erfasst und per Klick als erledigt markiert werden.
- **Arztbericht / PDF-Export (/berichte/):** Nach Auswahl eines Datumsbereichs generiert das System auf Knopfdruck einen druckoptimierten, mehrseitigen DIN-A4-Arztbericht flüchtig im Arbeitsspeicher (In-Memory) und stellt diesen als Download bereit.

2. Barrierefreier Patientenmodus / Tagesansicht (/patient/):

- **Fokus:** Diese Ansicht ist kognitiv stark reduziert, verzichtet auf Menüleisten und richtet sich direkt an die zu pflegende Person.
- **Bedienung:** Großflächige Kacheln zeigen die für heute fälligen Medikamente.
- **Einnahmequittierung:** Ein Klick auf „Als eingenommen markieren“ bucht die Einnahme in das revisionssichere Protokoll und dekrementiert synchron den physischen Lagerbestand. Bereits eingenommene Dosen werden dauerhaft grün und schreibgeschützt visualisiert.

Beenden der Anwendung: Der Webserver kann jederzeit im Konsolenfenster durch die Tastenkombination Strg + C gestoppt werden.

1.8.5 Replikation der Testsuite und Qualitätsprüfung

Ausführung aller 17 automatisierten Testfälle:

```
PowerShell  
  
pytest
```

Ausführung der Performanz- und Lasttests:

PowerShell

```
pytest tests/integration/test_performance.py -s
```

2 Anforderungsdokumentation

Das folgende Kapitel erfasst die fachlichen und qualitativen Anforderungen an das System. Neben der Analyse der beteiligten Stakeholder werden die funktionalen Anforderungen als User Stories formuliert sowie die nichtfunktionalen Qualitätskriterien nach ISO/IEC 25010 festgelegt.

2.1 Stakeholder-Analyse

Die Stakeholder-Analyse identifiziert alle Personen, Gruppen und Organisationen, die Einfluss auf das System haben oder von dessen Nutzung direkt oder indirekt betroffen sind. In Tabelle 3 werden die drei wesentlichen Stakeholder-Gruppen hinsichtlich ihrer Systemrolle, ihrer Erwartungshaltung sowie ihrer technischen Vorkenntnisse gegenübergestellt.

Tabelle 3 Stakeholder-Analyse

Stakeholder-Gruppe	Rolle im System	Bedürfnisse & Erwartungen	Technische Vorkenntnisse
Pflegende Angehörige	Hauptnutzende	• Schnelle Erfassung von Medikationsplänen und Vitaldaten • Terminerinnerungen und lückenlose Historie • Automatisierte Erstellung von PDF-Berichten für ärztliches Fachpersonal	Grundlegendes IT-Verständnis (Webbrowser / Smartphone)
Zu pflegende Personen	Endnutzende (Patientenansicht)	• Maximale Übersichtlichkeit und Barrierefreiheit • Große Schriften, hohe Kontraste und einfache Interaktion • Klare Fokus-Frage: „<i>Welche Medikamente muss ich jetzt einnehmen?</i>“	Geringe bis sehr geringe IT-Affinität
Ärztliches Fachpersonal	Passiv (empfangend)	• Übersichtliche Zusammenfassung von Vitalwerten (z. B. Blutdruckverläufe) • Nachvollziehbarkeit der Einnahmetreue (Compliance)	Keine direkte Systemnutzung (erhält ausgedrucktes/digitales PDF)

Quelle: eigene Darstellung

Aus dieser Gegenüberstellung wird deutlich, dass das System zwei stark differierende Nutzungskontexte bedienen muss: Während für pflegende Angehörige der Funktionsumfang und die effiziente Datenverwaltung im Vordergrund stehen, erfordert die Interaktion mit den zu pflegenden Personen eine strikte Reduktion der kognitiven Belastung.

2.2 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben das Soll-Verhalten der Anwendung aus Perspektive der jeweiligen Anwenderrollen. Zur lückenlosen Spezifikation der funktionalen Anforderungen dienen User Stories nach dem etablierten IREB-Standard (Pohl & Rupp, 2021). Diese werden im Hinblick auf das Minimum Viable Product (MVP) nach fachlicher Relevanz priorisiert.

FA-01: Medikationsverwaltung (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich Medikamente mit Name, Dosierung, Einnahmezeitpunkt und Bestand anlegen, bearbeiten und löschen, um den Medikationsplan stets aktuell zu halten.“

Fachlicher Kontext: Unvollständige oder veraltete Pläne führen im Pflegealltag zu Medikationsfehlern. Das System stellt strukturierte Eingabemasken bereit, die Pflichtfelder erzwingen.

Akzeptanzkriterien: Die Absendung des Formulars gelingt nur, wenn alle Pflichtfelder vollständig und im korrekten Datentyp ausgefüllt sind. Nach dem Speichern erscheint der neue Datensatz unverzüglich in der Medikationsübersicht. Änderungen oder Löschungen werden ohne Datenverlust synchron in der Datenbank reflektiert.

FA-02: Vitaldaten-Tracking (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich tägliche Vitalwerte (Blutdruck, Puls, Gewicht) mit Zeitstempel erfassen und in Tabellen einsehen, um gesundheitliche Trends frühzeitig zu erkennen.“

Fachlicher Kontext: Das Erfassen strukturierter Messwerte liefert dem behandelnden Fachpersonal essenzielle Anhaltspunkte für klinische Bewertungen.

Akzeptanzkriterien: Das System versieht jeden abgespeicherten Messwert automatisch mit dem aktuellen Server-Zeitstempel. Texteingaben in numerischen Feldern werden serverseitig abgefangen. Die erfassten Messwerte werden in der historischen Verlaufstabelle chronologisch absteigend sortiert dargestellt.

FA-03: Arzt-Bericht / PDF-Export (Priorität: Mittel)

User Story: „Als pflegende Person möchte ich gefilterte Vital- und Medikationsdaten für einen wählbaren Zeitraum als PDF exportieren, um diese beim Hausarztbesuch vorzulegen.“

Fachlicher Kontext: Arztbesuche leiden häufig unter Zeitmangel und unvollständigen Patientenauskünften. Der automatisierte Export aggregiert alle relevanten Historien auf Knopfdruck.

Akzeptanzkriterien: Die anwendende Person kann das Start- und Enddatum des Berichtszeitraums über einen Datumsfilter einschränken. Das generierte PDF-Dokument enthält Stammdaten, verordnete Medikamente sowie tabellarische Vitalwertverläufe in einem druckoptimierten Layout. Die Generierung und Bereitstellung des Downloads erfolgen innerhalb von unter zwei Sekunden.

FA-04: Termin- und Aufgabenverwaltung (Priorität: Mittel)

User Story: „Als pflegende Person möchte ich anstehende Pflegetermine sowie Aufgaben eintragen, um den Pflegealltag strukturiert zu organisieren.“

Fachlicher Kontext: Die Koordination von Arztbesuchen, Physiotherapie und pflegerischen Verpflichtungen erfordert eine verlässliche Zeitplanung.

Akzeptanzkriterien: Anstehende Termine werden chronologisch sortiert auf einem Aufgaben-Board dargestellt. Durch Anklicken einer Status-Checkbox lässt sich der Termin als erledigt kennzeichnen, woraufhin die visuelle Darstellung angepasst wird.

FA-05: Barrierefreie Tagesansicht / Patientenmodus (Priorität: Hoch)

User Story: „Als zu pflegende Person möchte ich eine vereinfachte Tagesansicht aufrufen können, die mir in großer Schrift nur die heute anstehenden Medikamente anzeigt, um Orientierung im Alltag zu erhalten.“

Fachlicher Kontext: Zur Förderung der Handlungsautonomie verzichtet diese Ansicht auf verschachtelte Menüstrukturen und administrative Bedienelemente.

Akzeptanzkriterien: Die Oberfläche enthält keine komplexen Navigationsleisten. Die Basis-Schriftgröße beträgt durchgängig mindestens 18 pt.

FA-06: Einnahmebestätigung (Priorität: Hoch)

User Story: „Als anwendende Person (pflegende oder zu pflegende Person) möchte ich eine anstehende Tagesdosis per Klick auf eine großflächige Schaltfläche als eingenommen markieren können, um den Vollzug der Medikation eindeutig zu dokumentieren.“

Fachlicher Kontext: Die explizite Entkopplung von der Medikationsverwaltung stellt sicher, dass das bloße Erfassen eines Medikaments vom tatsächlichen Einnahmeakt im Alltag getrennt bleibt.

Akzeptanzkriterien: Der Klick auf die Bestätigungsschaltfläche löst eine Statusänderung der spezifischen Einnahme für den aktuellen Tag aus. Das UI kennzeichnet die Kachel sofort als eingenommen inklusive des genauen Ausführungszeitstempels. Die Einnahme wird unveränderlich im Protokoll verankert.

FA-07: Automatische Bestandsaktualisierung & Schwellenwertprüfung (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich, dass der Systembestand eines Medikaments bei jeder Einnahmebestätigung automatisch dekrementiert wird und bei Unterschreiten des Mindestbestands eine Warnmeldung erscheint, um Versorgungsengpässe zu vermeiden.“

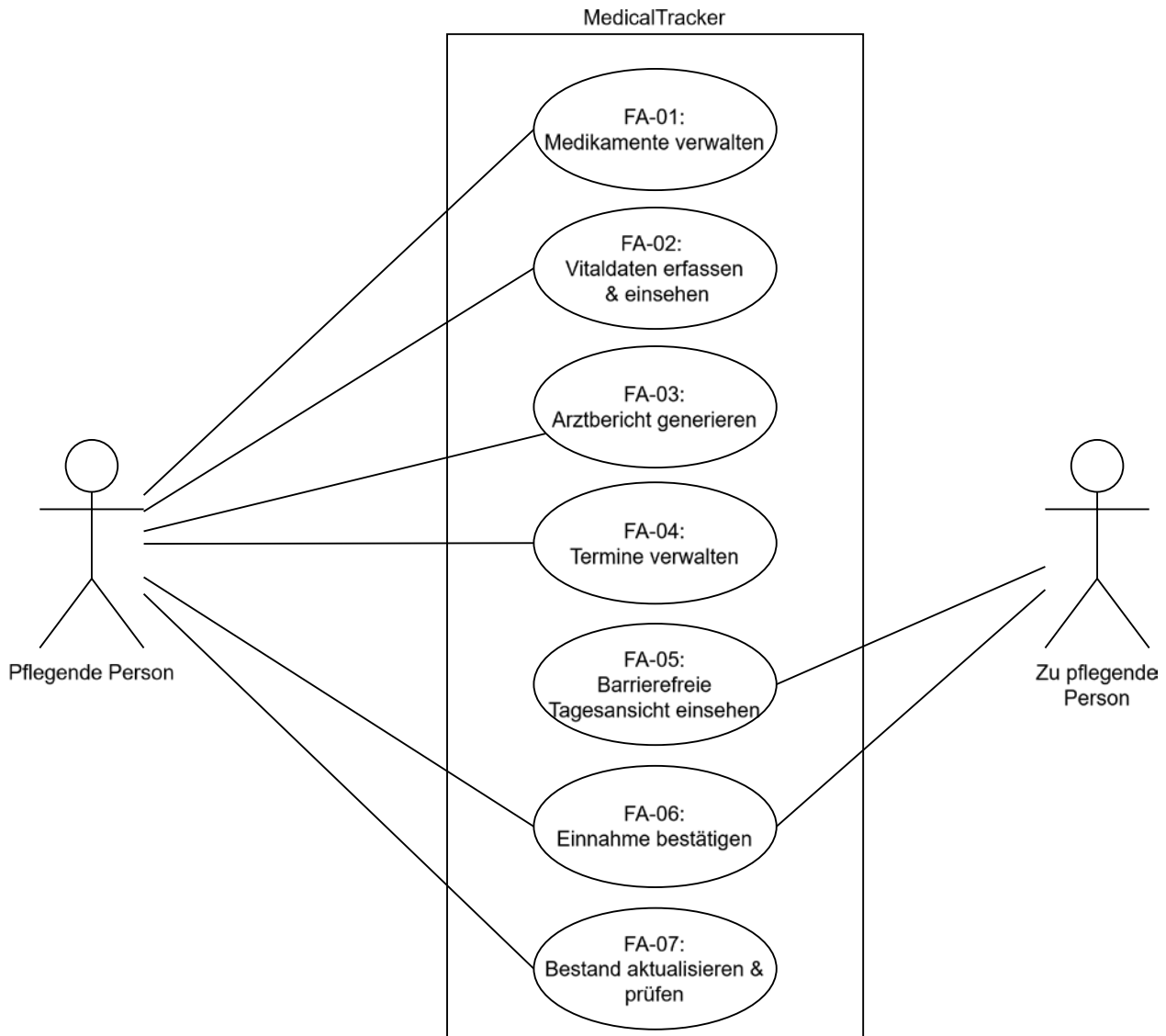
Fachlicher Kontext: Die automatisierte Koppelung von Einnahmeakt und Lagerbestand verhindert das unbemerkte Auslaufen wichtiger Arzneimittel.

Akzeptanzkriterien: Bei jeder erfolgreichen Ausführung von FA-06 reduziert das Backend den Wert des Attributs *bestand* um die verabreichte Einheitsmenge. Ergibt die anschließende Prüfung, dass $bestand \leq mindestbestand$ gilt, wird auf dem Dashboard der pflegenden Person unverzüglich ein optisch hervorgehobener Nachbestell-Warnhinweis generiert.

2.3 UML-Anwendungsfalldiagramm

Zur visuellen Abgrenzung der Systemgrenzen sowie zur Verdeutlichung der Interaktionen zwischen den identifizierten Akteuren und den funktionalen Anforderungen dient das in Abbildung 1 dargestellte UML-Anwendungsfalldiagramm.

Abbildung 1 Anwendungsfalldiagramm



Quelle: eigene Darstellung

Aus der Modellierung geht die funktionale Rollentrennung hervor: Während die pflegende Person als Hauptakteur Vollzugriff auf alle administrativen Verwaltungs- und Analysefunktionen (FA-01 bis FA-04, FA-06 und FA-07) besitzt, beschränkt sich die Interaktion der zu pflegenden Person gezielt auf den barrierefreien Patientenmodus (FA-05) sowie die Bestätigung der Einnahme (FA-06). Dies unterstützt das Ziel der kognitiven Entlastung der zu pflegenden Person.

2.4 Nichtfunktionale Anforderungen

Nichtfunktionale Anforderungen (NFA) definieren die qualitativen Eigenschaften, Sicherheitsgrenzen und Betriebsbedingungen des Gesamtsystems nach dem Standard ISO/IEC 25010:2023. Während funktionale Anforderungen festlegen, welche fachlichen Funktionen bereitgestellt werden, bestimmen die Qualitätsanforderungen, unter welchen Restriktionen und Gütekriterien dies geschieht.

Um die geforderten Qualitätsattribute nicht als isolierte technische Vorgaben zu formulieren, leitet sich jede NFA unmittelbar aus den in Abschnitt 2.1 analysierten Bedürfnissen und Risiken der Stakeholder ab. Die Anforderungen sind im Hinblick auf das MVP priorisiert und werden nachfolgend anhand ihrer Spezifikation, messbaren Akzeptanzmetriken und einer wissenschaftlich-technischen Rationale operationalisiert:

NFA-01: Nutzbarkeit / Barrierefreiheit (Kategorie: Interaction Capability - Priorität: Hoch)

Spezifikation & Stakeholder-Bezug:

Gestaltung des isolierten Patientenmodus (FA-05) speziell für zu pflegende Personen mit altersbedingten sensorischen und motorischen Einschränkungen nach den WCAG.

Messbare Akzeptanzmetrik:

Kontrastverhältnis zwischen Text und Hintergrund mindestens 7:1 (Konformitätsstufe WCAG 2.2 AAA). Basis-Schriftgröße im Patientenmodus durchgängig mindestens 18 pt. Interaktive Klick- und Touch-Schaltflächen mit einer Mindestgröße von 48 x 48 px.

Wissenschaftlich-technische Rationale:

Im Pflegekontext führen Veränderungen des Sehvermögens sowie motorische Defizite bei standardisierten Web-Oberflächen zu hohen Fehlerraten. Schriftgrade unter 18 pt und herkömmliche Kontraste (4,5:1) erhöhen die Verwechslungsgefahr bei Einnahmezeitpunkten und Wirkstoffnamen. Die Vorgabe von mindestens 48 x 48 px für Klickelemente hilft den Zielerfassungsaufwand zu minimieren und Fehlklicks verlässlich abzufangen.

NFA-02: Leistungsfähigkeit (Kategorie: Performance Efficiency - Priorität: Hoch)

Spezifikation & Stakeholder-Bezug:

Verzögerungsfreie Rückmeldung bei Interaktionen von Angehörigen und Patienten zur Vermeidung von Verunsicherungen über den Transaktionsstatus.

Messbare Akzeptanzmetrik:

Antwortzeit des lokalen Webserver beim Laden und Rendern von Ansichten maximal 1 Sekunde unter Windows 10/11. Ausführungszeit für Lese- und Schreibzugriffe auf die Datenbank unter 200 ms. Vollständige In-Memory-Generierung und Download-Bereitstellung des PDF-Arztberichts (FA-03) innerhalb von maximal 2 Sekunden.

Wissenschaftlich-technische Rationale:

Gemäß den etablierten Grundsätzen der Human-Computer Interaction (Nielsen, 1993) markiert eine Reaktionszeit von 1 Sekunde die Obergrenze, bis zu der ein Benutzer einen kontinuierlichen Gedankengang ohne störende Unterbrechung wahrnimmt. Im Pflegealltag erzeugen Verzögerungen bei der Einnahmequittierung Zweifel darüber, ob die Interaktion vom System registriert wurde, was fehlerhafte Doppelbuchungen provoziert. Das Latenzbudget von unter 200 ms für Datenbank-Transaktionen stellt sicher, dass die synchrone Bestandsdekrementierung (FA-07) abgeschlossen ist, bevor das UI den Erfolgsstatus visualisiert.

NFA-03: Sicherheit (Kategorie: Security - Priorität: Hoch)

Spezifikation & Stakeholder-Bezug:

Lückenloser Schutz der verarbeiteten Gesundheitsdaten (besondere Kategorien nach Art. 9 DSGVO) vor unbefugtem Zugriff, Datenabfluss und Manipulation für alle Stakeholder.

Messbare Akzeptanzmetrik:

100 % lokale Persistenz auf dem Host-System; vollständiger Ausschluss von Telemetrie, Cloud-Backends oder externen API-Verbindungen (Privacy by Design). 0 Byte unverschlüsselte temporäre PDF-Zwischendateien im lokalen Dateisystem durch In-Memory-Verarbeitung. Vollständige Immunität gegen SQL-Injection durch parametrisierte Bindings des ORMs sowie gegen Cross-Site-Scripting (XSS) durch kontextsensitives Auto-Escaping. Kryptographischer CSRF-Tokenschnittstellen-Schutz auf allen modifizierenden POST-Routen.

Wissenschaftlich-technische Rationale:

Gesundheitsdaten unterliegen nach Art. 9 Abs. 1 DSGVO einem gesteigerten Schutzniveau. Eine externe Cloud-Speicherung würde administrative Pflichten wie eine Auftragsverarbeitung (Art. 28 DSGVO) oder eine Datenschutz-Folgenabschätzung auslösen. Derartige Auflagen können private Anwender im häuslichen Umfeld nicht rechtssicher erfüllen. Die lokale Datenhaltung verankert das System im Schutzbereich der DSGVO-Haushaltsausnahme (Art. 2 Abs. 2 lit. c). Die flüchtige In-Memory-Kompilierung von PDF-Dokumenten verhindert das persistente Verweilen ungeschützter Patientendaten im Betriebssystem-Cache.

NFA-04: Zuverlässigkeit (Kategorie: Reliability - Priorität: Hoch)

Spezifikation & Stakeholder-Bezug:

Robuste Fehlertoleranz und absolute Transaktionskonsistenz bei Fehlbedienungen durch pflegende Angehörige oder Fehleingaben.

Messbare Akzeptanzmetrik:

Keine unbehandelte Serverabstürze (HTTP 500) bei syntaktisch oder semantisch ungültigen Formularwerten (z. B. Text im Blutdruckfeld oder unplausible Datumsangaben). Automatisches

Transaktions-Rollback bei jeglicher Datenbank-Exception. Eindeutige, feldbezogene Fehlerquittierung an der UI-Komponente mit konkretem Korrekturhinweis.

Wissenschaftlich-technische Rationale:

Im medizinischen Dokumentationskontext bedeutet ein Programmabsturz bei fehlerhaften Eingaben einen potenziell irreversiblen Datenverlust und gefährdet die lückenlose Verlaufsdokumentation. Das transaktionale Rollback sichert das ACID-Prinzip ab und stellt sicher, dass keine Halbzuteilungen (z. B. Bestätigung der Einnahme im Protokoll, aber Abbruch der Bestandsdekrementierung) in der Datenbank verbleiben.

NFA-05: Flexibilität / Portabilität (Kategorie: Portability - Priorität: Hoch)

Spezifikation & Stakeholder-Bezug:

Plattformunabhängige, installationsfreie Inbetriebnahme im privaten Haushalt ohne Notwendigkeit administrativer IT-Kenntnisse seitens der Angehörigen.

Messbare Akzeptanzmetrik:

Autarke Lauffähigkeit im Standard-Webbrowser unter Microsoft Windows 10 und Windows 11 (64-Bit). Als Laufzeitumgebung ist ein Standard-Python-Interpreter hinreichend. Vollständiger Verzicht auf externe Hintergrunddienste, Daemon-Prozesse oder dedizierte RDBMS-Dienste (Zero-Configuration-Deployment).

Wissenschaftlich-technische Rationale:

Pflegende Angehörige verfügen typischerweise über reguläre Anwenderkompetenzen, nicht jedoch über Kenntnisse zur Konfiguration relationaler Client-Server-Datenbanksysteme (wie MySQL oder PostgreSQL). Die Wahl einer prozessintegrierten SQLite-Datendatenbank in Kombination mit dem integrierten Flask-WSGI-Server ermöglicht eine Inbetriebnahme über einen einfachen Skriptaufruf, ohne Systemports oder Dienstprivilegien manipulieren zu müssen.

NFA-06: Wartbarkeit & Testbarkeit (Kategorie: Maintainability - Priorität: Hoch)

Spezifikation & Stakeholder-Bezug:

Strukturierte Codebasis nach dem MVC-Muster zur Sicherstellung der Weiterentwickelbarkeit und Verifizierbarkeit durch Entwickler und Prüfungsgremien.

Messbare Akzeptanzmetrik:

Modulare Entkopplung der Steuerungslogik nach fachlichen Domänen über Flask-Blueprints. Konsequente Einhaltung des PEP-8-Standards mit einem PyLint-Score von mindestens 9,0/10. Automatisierte Testabdeckung (Unit- und Integrationstests via pytest) der Kern-Geschäftslogik (Validatoren, Bestandsberechnungen, Geschäftsregeln GR-01 bis GR-04) von mindestens 80 % Line Coverage.

Wissenschaftlich-technische Rationale:

Pflegeprozesse und physiologische Leitlinien (z. B. Blutdruckschwellen nach ESC) unterliegen regelmäßigen fachlichen Revisionen. Die Entkopplung nach MVC und das Blueprint-Pattern garantieren, dass funktionale Erweiterungen ohne Rückwirkungen auf die Persistenzschicht erfolgen können. Eine automatisierte Testabdeckung von mehr als 80 % sichert die Regressionsfreiheit des Gesamtsystems bei Refactorings dauerhaft ab. Die Begrenzung der McCabe-Komplexität stellt sicher, dass der Kontrollfluss verständlich und auditierbar bleibt.

NFA-07: Skalierbarkeit (Kategorie: Scalability – Priorität: Mittel)

Spezifikation & Stakeholder-Bezug:

Performanzstabilität über mehrjährige Pflegezeiträume für Patient und Angehörige sowie Vorbereitung auf einen erweiterten Mehrpersonenbetrieb.

Messbare Akzeptanzmetrik:

Verzugfreie Verarbeitung und Abfrage von mindestens 10.000 historischen Datensätzen (Vitaldaten und Einnahmeprotokolle) pro Patient ohne Verletzung der Antwortzeitbudgets aus NFA-02. Relationales Datenbankschema mit indizierten Fremdschlüsseln zur Vermeidung von Full-Table-Scans bei Verknüpfungen. Migrationsfähigkeit auf ein vollwertiges Client-Server-RDBMS (z. B. PostgreSQL) ohne Anpassung des Domänenencodes durch die ORM-Abstraktion.

Wissenschaftlich-technische Rationale:

Chronische Pflegeverläufe erstrecken sich typischerweise über mehrere Jahre. Bei täglich 3–4 Einnahmen und 2 Vitalwertmessungen entstehen jährlich rund 2.000 bis 2.500 Datensätze. Der Richtwert von 10.000 Datensätzen bildet somit einen typischen 4- bis 5-Jahres-Horizont verlässlich ab. Die Abstraktion über SQLAlchemy entkoppelt die Anwendungslogik von SQLite-Spezifika, sodass bei künftiger Ausweitung des Betriebsmodus ein unterbrechungsfreier Austausch der Datenbankverbindung über die Konfiguration genügt.

2.5 Datenschutz- und Sicherheitsanalyse

Die Verarbeitung von Gesundheitsdaten unterliegt aufgrund ihres sensiblen Charakters strengen rechtlichen und technischen Anforderungen. Im bisherigen Softwaredesign wurde die Aussage der Datenschutzkonformität vorrangig auf die lokale Datenhaltung zurückgeführt. Eine wissenschaftlich fundierte Betrachtung erfordert jedoch die explizite Differenzierung der rechtlichen Grundlagen nach der DSGVO, die konkrete Spezifikation technischer und organisatorischer Maßnahmen sowie die Analyse der Hürden für einen potenziellen klinischen Realeinsatz.

2.5.1 Einordnung nach DSGVO

Gesundheitsdaten wie verordnete Medikamente, Einnahmeprotokolle und Vitalwerte stellen besondere Kategorien personenbezogener Daten im Sinne von Art. 9 Abs. 1 DSGVO dar. Die Zulässigkeit der Verarbeitung stützt sich auf zwei Säulen:

Die rechtliche Privat- und Haushaltsausnahme (Art. 2 Abs. 2 lit. c DSGVO)

Da die Anwendung ausschließlich für den privaten Gebrauch durch pflegende Angehörige und zu pflegende Personen innerhalb eines Haushalts konzipiert ist, greift primär die sogenannte Haushaltsausnahme. Diese entbindet rein persönliche oder familiäre Tätigkeiten von den administrativen Pflichten der DSGVO (wie dem Führen eines Verzeichnisses). Die strikte Vermeidung jeglicher Datenübertragung an externe Server oder Cloud-Dienste stellt sicher, dass dieser rechtliche Schutzraum nicht verlassen wird.

Explizite Einwilligung und Datenminimierung (Art. 6 Abs. 1 lit. a & Art. 9 Abs. 2 lit. a DSGVO)

Sollten Daten im häuslichen Umfeld zwischen der pflegenden und der zu pflegenden Person ausgetauscht werden, erfolgt dies auf Basis einer freiwilligen, expliziten Einwilligung. Das System setzt das Prinzip der Datenminimierung (Privacy by Design) konsequent um: Es werden ausschließlich diejenigen Attribute verarbeitet, die für den Zweck der Pflegedokumentation zwingend erforderlich sind.

2.5.2 Technische und organisatorische Schutzmaßnahmen

Die reine Zusage einer lokalen Speicherung reicht zur Gewährleistung der Datensicherheit nicht aus, da auch lokale Systeme Bedrohungen wie unbefugtem physischen Zugriff, Schadsoftware oder Datenverlust ausgesetzt sind. Das Sicherheitskonzept spezifiziert daher konkrete Schutzmaßnahmen gemäß Art. 32 DSGVO:

Vertraulichkeit & Zugriffsschutz: Auf Anwendungsebene kapselt der Objekt-Relationale Mapper Flask-SQLAlchemy alle Datenbankzugriffe ab. Dies verhindert SQL-Injection-Angriffe zuverlässig und stellt sicher, dass ausschließlich typisierte, validierte Objekte persistiert werden. Das Betriebssystem schützt die SQLite-Datenbankdatei vor unbefugtem Auslesen. Hierzu setzt das Konzept auf lokale Zugriffsrechte (User Account Control) und empfiehlt eine Festplattenverschlüsselung (z. B. via Windows BitLocker).

Integrität & Eingabesicherheit: Um Angriffe über die Benutzeroberfläche auszuschließen, kommen client- und serverseitige Filtermechanismen zum Einsatz. Das automatische Escaping der

Jinja2-Template-Engine unterbindet Cross-Site-Scripting (XSS). Formulareingaben werden mittels CSRF-Tokens (Cross-Site Request Forgery via Flask-WTF) gegen ungewollte Anfragen geschützt.

Verfügbarkeit & Belastbarkeit: Zur Absicherung gegen Datenkorruption oder plötzliche Systemabstürze führt die Anwendung bei jedem Start eine automatisierte Integritätsprüfung des Dateikopfs der SQLite-Datenbank durch. Gleichzeitig wird vor dem Schreibzugriff eine automatisierte lokale Sicherheitskopie der Datenbankdatei in einem isolierten Backup-Ordner angelegt.

Evaluierung & Transaktionssicherheit: Fehlerhafte Transaktionen lösen einen sofortigen Rollback-Befehl aus, wodurch der Datenbankzustand bei Validierungsfehlern konsistent bleibt.

2.5.3 Anforderungen an einen potenziellen klinischen Realeinsatz

Sollte der MedicalTracker aus dem privaten Umfeld in den professionellen klinischen Betrieb (z. B. in Pflegediensten oder Arztpraxen) überführt werden, ergeben sich zusätzliche rechtliche und architektonische Anforderungen:

1. Zertifizierung als Medizinprodukt (MDR 2017/745): Das System müsste ein Konformitätsbewertungsverfahren durchlaufen, um die Anforderungen an Risikomanagement (ISO 14971) und Software-Lebenszyklusprozesse (IEC 62304) nachzuweisen.
2. Kryptographische Verschlüsselung (At-Rest & In-Transit): Die SQLite-Datenbank müsste auf Dateiebene verschlüsselt werden. Für Schnittstellen zu Praxissystemen wäre eine Ende-zu-Ende-Verschlüsselung nach TLS 1.3 zwingend erforderlich.
3. Rollennavigations- & Audit-Trail-System: Es müsste eine differenzierte Rechte- und Rollenmatrix (RBAC) mit strikter Trennung zwischen Pflegepersonal, ärztlichem Fachpersonal und zu pflegenden Personen implementiert werden. Sämtliche Lese- und Schreibzugriffe müssten in einem revisionssicheren Logbuch (Audit Trail) protokolliert werden.
4. Standardisierte Gesundheitsschnittstellen: Der Datenaustausch müsste über internationale Standards der Medizininformatik wie HL7 FHIR (Fast Healthcare Interoperability Resources) abgewickelt werden, um eine Interoperabilität mit der elektronischen Patientenakte (ePA) herzustellen.
5. Datenschutz-Folgenabschätzung (DSFA nach Art. 35 DSGVO): Vor Inbetriebnahme im klinischen Kontext wäre eine systematische Risikofolgenabschätzung unter Einbeziehung von Datenschutzbeauftragten gesetzlich vorgeschrieben.

2.6 Requirements-Traceability-Matrix

Die nachfolgende Requirements-Traceability-Matrix (Tabelle 4) stellt die durchgängige Vorwärts- und Rückwärtsverfolgbarkeit von den Stakeholder-Bedürfnissen über die Systemspezifikation und die MVC-Architektur bis hin zur Verifikation in Kapitel 5 auf einer kompakten Übersichtsseite her.

Tabelle 4 Requirements-Traceability-Matrix

Anforderung	Stakeholder& Kernziel	Spezifikation (Prozess / Regel)	Technische Umsetzung (MVC & Artefakte)	Verifikation (Test-ID)
FA-01 Medikations- verwaltung	Angehörige: Aktueller Medikationsplan; Ausschluss von Dosierungsfehlern	GP-01 (Erfassung) GR-01 (Bestand ≥ 0)	M: medikament.py C: medikamente.py V: medikamente/form.html	TC-FA-01-01 TC-GR-01-01
FA-02 Vitaldaten-Tra- cking	Angehörige / Arzt: Tägliche Verlaufsreihen; Erkennung physiologischer Trends	GP-02 (Erfassung) GR-02 (Plausibilität)	M: vitalwert.py C: vitaldaten.py V: vitaldaten/form.html	TC-FA-02-01 TC-GR-02-01
FA-03 Arzt-Bericht (PDF)	Arzt / Angehörige: Valide Entscheidungsgrundlage; Zeitersparnis bei Konsultation	Schnittstelle PDF; Filter: start < end	S: pdf_generator.py C: berichte.py V: berichte/index.html	TC-FA-03-01 TC-NFA-02-02
FA-04 Terminverwal- tung	Angehörige: Pflegetermine strukturieren; Entlastung im Zeitmanagement	Sortierung chronologisch; Status-Flag erledigt	M: termin.py C: termine.py V: termine/index.html	TC-FA-04-01
FA-05 Tagesansicht	Patient: Kognitiv reduzierte Orientierung; Erhalt der Selbstständigkeit	WCAG 2.2 Kriterien; Fokus: „Was steht an?“	V: tagesansicht.html CSS: patient.css (Kontrast) C: patient.py	TC-FA-05-01 TC-NFA-01-01
FA-06 Einnahmebe- stätigung	Patient / Angehörige: Compliance-Nachweis; revisions-sichere Dokumentation	GP-03 (Kernprozess); GR-03 (Historiensicherheit)	M: einnahme.py C: patient.py (POST einnehmen) JS: patient.js	TC-FA-06-01 TC-GR-03-01
FA-07 Bestandsfüh- rung	Angehörige: Unterbrechungsfreie Versorgung; Schutz vor Arzneimittelab-riss	GP-03 (Kernprozess); Prüfung: bestand $\leq$ min	M: medikament.py (bestand) C: Atomarer Commit (patient.py) V: Warn-Badge dashboard.html	TC-FA-07-01 TC-GR-01-02
NFA-01 Nutzbarkeit	Patient: Barrierefreie Lesbarkeit trotz sensorischer/motorischer Defizite	WCAG 2.2; ISO/IEC 25010	Kontrast $\geq 7:1$; Schrift ≥ 18 pt; Klickflächen $\geq 48 \times 48$ px	TC-NFA-01-01 (DOM-Inspek-tion)
NFA-02 Leistungsfähig- keit	Angehörige / Patient: Reibungslose Bedienung ohne Wartezeiten im Alltag	Ladezeit für UI, SQLite & PDF-Render	View-Ladezeit $\leq 1,0$ s; DB ≤ 200 ms; PDF-Generierung $\leq 2,0$ s	TC-NFA-02-01 (Zeitmessung)
NFA-03 Sicherheit	Alle: Schutz sensibler Gesundheitsdaten vor unbefugtem Zugriff	Art. 9 DSGVO; Privacy by Design	SQLite lokal; ORM Parameter-Binding; In-Memory-PDF (io.BytesIO)	TC-NFA-03-01 (Input Fuz-zing)
NFA-04 Zuverlässigkeit	Angehörige: Fehlertoleranz; Ausschluss von Abstürzen bei Falscheingaben	Plausibilitätsfenster; Fehlerquittierung	WTForms-Validierungskette; Transaktionales Rollback bei Fehler	TC-NFA-04-01 (Negativtests)
NFA-05 Flexibilität	Angehörige: Einfache Inbetriebnahme ohne dedizierten Datenbankserver	Standard-Windows-10/11; Zero-Configuration	Autarker Flask-WSGI-Server auf localhost; Python 3.14+; SQLite-Datei	TC-NFA-05-01 (Systemtest Win)
NFA-06 Wartbarkeit	Entwickler / Prüfer: Modulare, erweiterbare und verständliche Codebasis	MVC-Architektur; PEP-8 Standard	Flask-Blueprints; PyLint $\geq 9,0$; Testabdeckung Logik $\geq 80\%$	TC-NFA-06-01 (pytest-cov)
NFA-07 Skalierbarkeit	Angehörige / Arzt: Performanz über mehrjährige Dokumentationszeiträume	Datenhaltung für chronische Verläufe	Relationales Schema; Fremdschlüssel-Indizierung; bis 10.000 Datensätze	TC-NFA-07-01 (Volumentest)

Quelle: eigene Darstellung

3 Spezifikationsdokumentation

Dieses Kapitel beschreibt das objektorientierte Datenmodell der zentralen Geschäftsobjekte sowie deren fachliche Beziehungen und Kardinalitäten. Des Weiteren werden die dynamischen Geschäftsprozesse – mit besonderem Fokus auf den Kernprozess der Medikamenteneinnahme und Bestandsführung – detailliert dargelegt. Ergänzend werden die strikt einzuhaltenden Geschäftsregeln formuliert sowie die technischen System- und Benutzerschnittstellen spezifiziert.

3.1 Objektorientiertes Datenmodell & Geschäftsobjekte

Das Datenmodell bildet das statische Fundament der Anwendung und strukturiert die zentralen Geschäftsobjekte sowie deren relationale Verknüpfungen. Zur Behebung struktur- und ablauftechnischer Unschärfen wird die Entität der Einnahmeprotokollierung explizit im Schema verankert und in ihrer relationalen Beziehung zu den Stammdaten der Medikation und des Patienten präzisiert.

1. Geschäftsobjekt: Patient

Fungiert als primärer Ankerpunkt des relationalen Modells. Über den Primärschlüssel *id* stehen alle personenbezogenen Gesundheitsdaten in einer eindeutigen 1:n-Beziehung zum Patienten.

- *id* (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- *vorname* (String): Vorname der zu pflegenden Person.
- *nachname* (String): Nachname der zu pflegenden Person.
- *geburtsdatum* (Date): Geburtsdatum.
- *notfallkontakt* (String): Telefonnummer oder Name einer Kontaktperson für Notfälle.

2. Geschäftsobjekt: Medikament

Repräsentiert die verordneten Arzneimittel und verweist über den Fremdschlüssel *patient_id* auf den zugehörigen Patienten. Während die Entität Medikament den aktuellen Soll-Zustand und den verfügbaren Gesamtvorrat beschreibt, erfordert die lückenlose Nachvollziehbarkeit des Ist-Zustands eine dedizierte Protokollentität.

- *id* (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- *patient_id* (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- *name* (String): Handelsname des Medikaments (z. B. "Ramipril 5mg").
- *dosierung* (String): Verordnete Einnahmemenge (z. B. "1 Tablette").
- *tageszeit* (String): Vorgesehener Einnahmezeitpunkt (z. B. "Morgens").
- *bestand* (Integer): Aktuell vorhandener Vorrat in Einheiten.
- *mindestbestand* (Integer): Schwellenwert für automatische Nachbestell-Warnungen.

3. Geschäftsobjekt: EinnahmeProtokoll

Verknüpft das statische Medikamentenobjekt mit dynamischen Ausführungsereignissen im Pflegealltag. Jedes Protokollobjekt besitzt einen eigenen Primärschlüssel *id* sowie die zwei Fremdschlüssel *patient_id* und *medikament_id*. Durch diese Modellierung bleibt die historische Dokumentation auch dann unverändert erhalten, wenn Stammdaten des Ursprungsmedikaments angepasst oder gelöscht werden.

- `id` (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- `patient_id` (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- `medikament_id` (Integer, Foreign Key): Referenz auf das Medikament.
- `soll_tageszeit` (String): Geplante Einnahmezeit.
- `einnahme_zeitpunkt` (DateTime): Tatsächliche Einnahmezeit.
- `erledigt` (Boolean): Boolescher Flag, ob die Einnahme erfolgt ist.

4. Geschäftsobjekt: Vitalwert

Erfasst eine einzelne medizinische Messung zu einem konkreten Zeitpunkt und verweist über den Fremdschlüssel *patient_id* auf den zentralen Patienten.

- `id` (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- `patient_id` (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- `zeitpunkt` (DateTime): Datum und Uhrzeit der Messung.
- `kategorie` (String): Art der Messung (z. B. "Blutdruck Systolisch", "Puls", "Gewicht").
- `wert` (Float): Gemessener numerischer Wert.
- `einheit` (String): Maßeinheit des Werts (z. B. "mmHg", "bpm", "kg").

5. Geschäftsobjekt: Termin

Speichert geplante Pflegetermine und Aufgaben im Alltagsplaner und verweist ebenfalls über den Fremdschlüssel *patient_id* auf den zentralen Patienten.

- `id` (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- `patient_id` (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- `titel` (String): Kurzbeschreibung des Termins (z. B. "Arzttermin Dr. Weber").
- `zeitpunkt` (DateTime): Ansetzungszeitpunkt.
- `kategorie` (String): Art des Termins (z. B. "Arzt", "Physiotherapie").
- `erledigt` (Boolean): Bearbeitungsstatus (Ja/Nein).

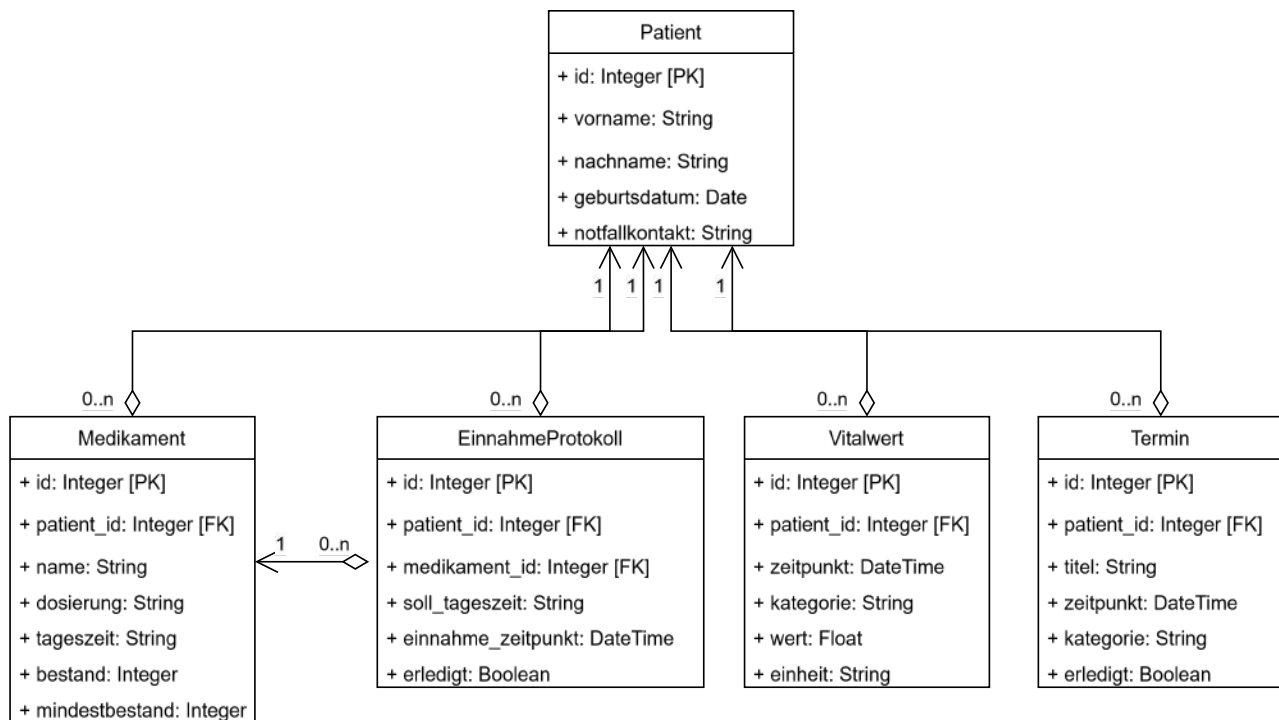
Multiplizitäten & Fachliche Beziehungen

Zwischen den Geschäftsobjekten bestehen klare 1:n-Kardinalitäten:

- Ein Patient besitzt beliebig viele zugeordnete Medikamente.
- Ein Patient besitzt beliebig viele Einnahmen.
- Ein Medikament besitzt beliebig viele Einnahmen.
- Ein Patient besitzt beliebig viele historische Vitalwerte.
- Ein Patient besitzt beliebig viele geplante Termine.

Die statische Struktur des Datenmodells sowie die fachlichen Beziehungsstrukturen und Kardinalitäten zwischen den zentralen Geschäftsobjekten werden im nachfolgenden UML-Klassendiagramm (Abbildung 2) zusammenfassend abgebildet.

Abbildung 2 Klassendiagramm



Quelle: eigene Darstellung

Das Klassendiagramm verdeutlicht die zentrale Stellung des Geschäftsobjekts Patient. Die abhängigen Entitäten Medikament, Vitalwert und Termin stehen jeweils über fremdschlüsselbasierte 1:n-Kardinalitäten in direkter Relation zum Patienten. Diese Struktur garantiert, dass alle Gesundheits- und Pflegedaten im System eindeutig einer Person zugeordnet sind.

3.2 Geschäftsprozesse

Die Geschäftsprozesse beschreiben das dynamische Zusammenspiel zwischen Benutzereingaben, Systemlogik und Datenhaltung.

GP-01: Erfassung eines neuen Medikaments

Der Prozess beginnt mit dem Aufruf der Medikationsübersicht durch den pflegenden Angehörigen. Nach Klick auf die Schaltfläche „Neues Medikament“ öffnet sich ein Formular. Die anwendende Person gibt die Stammdaten (Name, Dosierung, Einnahmezeitpunkt) sowie den aktuellen Anfangsbestand und den gewünschten Mindestbestand ein. Bei der Übermittlung prüft das System die Eingaben auf Vollständigkeit und numerische Plausibilität. Nach erfolgreicher Validierung wird der Datensatz persistent in der SQLite-Datenbank gespeichert und die anwendende Person erhält eine Bestätigungsmeldung.

GP-02: Vitaldatenerfassung und Schwellenwertprüfung

Zur Dokumentation von Vitalwerten wählt die anwendende Person die entsprechende Kategorie (z. B. Blutdruck) aus. Nach der Werteingabe führt das System eine client- und serverseitige Validierung durch, um fehlerhafte Datentypen (z. B. Texteingaben) abzufangen. Ergänzend wird geprüft, ob die eingegebenen Werte innerhalb medizinisch plausibler Grenzen liegen. Liegt beispielsweise der systolische Blutdruckwert unter dem diastolischen Wert, bricht das System den Speichervorgang ab und gibt eine detaillierte Fehlermeldung aus. Validierte Werte werden zusammen mit dem aktuellen Server-Zeitstempel in der Historie abgelegt.

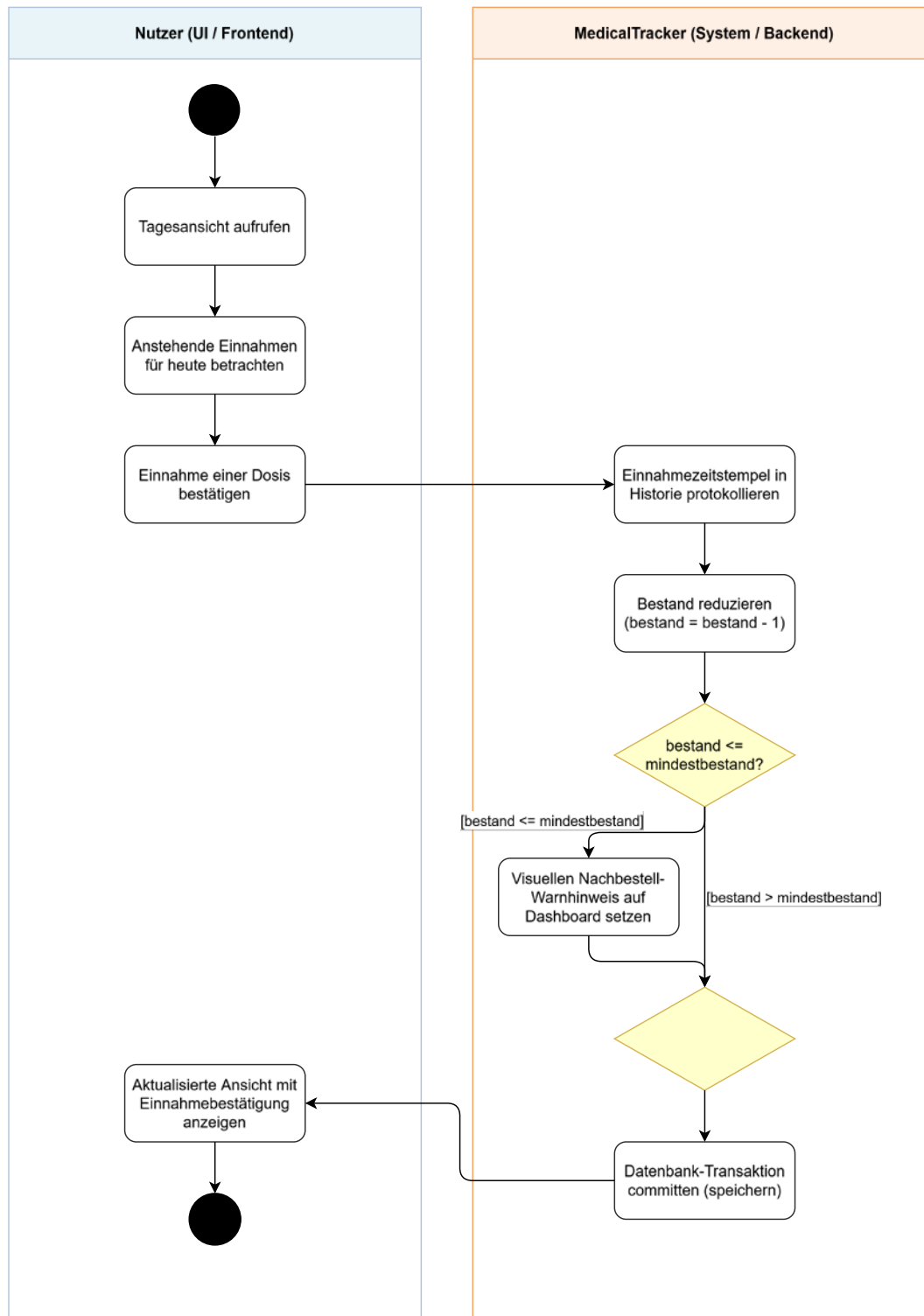
GP-03: Kernprozess - Medikamenteneinnahme und Bestandsaktualisierung

Der zentrale Geschäftsprozess verknüpft die alltägliche Pflegeinteraktion direkt mit der automatisierten Bestandsführung des Systems. Das Ablaufmuster gliedert sich in folgende Schritte:

1. **Prozessstart:** Die pflegende oder zu pflegende Person öffnet die barrierefreie Tagesansicht. Das System liest die für den aktuellen Tag terminierten Einnahmen aus der Datenbank aus und stellt diese dar.
2. **Einnahmebestätigung:** Die anwendende Person markiert eine anstehende Dosis durch Anklicken der entsprechenden Bestätigungsschaltfläche als „eingenommen“.
3. **Zustandsänderung & Transaktionsstart im Backend:** Das Backend nimmt den HTTP-POST-Request entgegen und initiiert eine atomare Datenbanktransaktion. Im selben Transaktionskontext werden zwei synchrone Operationen ausgeführt:
 - a. **Protokollerstellung:** Es wird ein neues Objekt der Entität EinnahmeProtokoll instanziiert, mit dem aktuellen Server-Zeitstempel sowie den Fremdschlüsseln versehen.
 - b. **Bestandsdekrementierung:** Das zugehörige Medikament-Objekt wird geladen und dessen Attribut bestand um die verabreichte Dosis reduziert.
4. **Verzweigung & Schwellenwertprüfung:** Das System führt eine logische Verzweigung durch und prüft die Geschäftsregeln (GR-01 Bestandspositivität). Ist der Schwellenwert unterschritten, wird ein visueller Nachbestell-Warnhinweis auf dem Dashboard gesetzt.
5. **Transaktionsabschluss & Rollback:** Verläuft die Validierung fehlerfrei, wird die Transaktion dauerhaft in der Datenbank gespeichert. Schlägt ein Teilschritt fehl, führt das System ein Rollback aus. Dies verhindert eine Diskrepanz zwischen digitalem Protokoll und physischem Medikamentenbestand.

Das UML-Aktivitätsdiagramm (Abbildung 3) veranschaulicht den dynamischen Ablauf des Kernprozesses GP-03 samt Kontrollflüssen und Entscheidungslogiken. Die Unterteilung in die Swimlanes Frontend (Nutzer) und Backend (System) grenzt die Zuständigkeiten klar ab.

Abbildung 3 Aktivitätsdiagramm



Quelle: eigene Darstellung

Wie die Prozessmodellierung zeigt, koppelt das System die Interaktion im Frontend direkt mit einer automatisierten Bestandskontrolle im Backend. Durch die Auswertung der Bedingung wird sichergestellt, dass bei kritischen Beständen unverzüglich eine optische Nachbestell-Warnung auf dem Dashboard generiert wird, bevor die Datenbanktransaktion erfolgreich abgeschlossen wird.

3.3 Geschäftsregeln

Geschäftsregeln definieren fachliche Bedingungen und Invarianten, die vom System jederzeit durchgesetzt werden müssen. Prüfroutinen im Frontend, Validierungslogiken im Backend und Constraints in der SQLite-Datenbank sichern die Regeln ab. Dieses mehrstufige Validierungskonzept verhindert Fehleingaben und gewährleistet eine verlässliche Datenbasis für das Pflegepersonal. Dies umfasst Prüfroutinen auf der Benutzeroberfläche, Validierungslogiken in den Flask-Routen sowie harte Integritätsbedingungen innerhalb des SQLite-Datenbankschemas. Nachfolgend werden die zentralen Geschäftsregeln des Systems detailliert spezifiziert:

GR-01 (Bestandspositivität):

Formale Bedingung:

$$\text{bestand} \geq 0 \wedge \text{mindestbestand} \geq 0$$

Begründung:

Ein negativer Arzneimittelbestand ist physikalisch unmöglich. Negative Werte würden die automatisierte Nachbestell-Logik verfälschen und das Generieren von Warnmeldungen auf dem Dashboard blockieren oder fehlerhafte Signale auslösen.

Technische Durchsetzung:

- Clientseitig: HTML5-Eingabefelder mit dem Attribut `min="0"`.
- Serverseitig: Validierung vor Ausführung des Datenbank-Commits.
- Datenbankebene: Verankerung einer Prüfung ('bestand >= 0') im SQLAlchemy-Modell der Entität Medikament.

GR-02 (Plausibilität der Vitaldatenerfassung):

Formale Bedingung:

$$\text{Blutdruck: } \text{systolisch} > \text{diastolisch} \wedge (30 \text{ mmHg} \leq \text{diastolisch} < \text{systolisch} \leq 300 \text{ mmHg})$$

$$\text{Pulsfrequenz: } 30 \text{ bpm} \leq f_{\text{puls}} \leq 250 \text{ bpm}$$

$$\text{Körpergewicht: } 2,0 \text{ kg} \leq m \leq 300,0 \text{ kg}$$

Begründung:

Aus physiologischer Sicht beschreibt der systolische Wert den maximalen Druck in der Kontraktionsphase des Herzens, wogegen der diastolische Wert den Dauerdruck in der Entspannungsphase wiedergibt (McEvoy et al., 2024). Ein Vertauschen beider Werte würde die automatische Verlaufsanalyse für das behandelnde Fachpersonal verfälschen.

Die Schranken von 30 mmHg bis 300 mmHg bilden ein softwareseitiges Validierungsfenster. Dieses deckt extreme klinische Zustände von schweren Schockzuständen bis hin zu hypertensiven Krisen ab. Die Grenzwerte orientieren sich an den technischen Messbereichen für nicht-invasive Blutdruckmessgeräte nach DIN EN ISO 81060-2 (Deutsches Institut für Normung, 2024). Werte außerhalb dieses Fensters weisen mit an Sicherheit grenzender Wahrscheinlichkeit auf Fehleingaben (z. B. Tippfehler) hin. Für die Pulsfrequenz orientiert sich das Intervall an den medizinischen Extrema (von extremer Bradykardie bis hin zu paroxysmalen Tachykardien) gemäß den ESC-Leitlinien (Glikson et al., 2021) und der Gerätenorm DIN EN ISO 80601-2-61:2019. Die Grenzen für das Körpergewicht definieren ein softwareseitiges Validierungsfenster zur Vermeidung von Fehleingaben (z. B. Kommasetzungsfehler). Das Intervall deckt das gesamte Spektrum medizinischer Waagen gemäß DIN EN 45501:2015 (Eichklasse III) ab.

Technische Durchsetzung:

- Serverseitig: Eine benutzerdefinierte Validierungsfunktion im Flask-Backend prüft bei der Übermittlung die Werte.
- Fehlerbehandlung: Bei Regelverletzung wird die Transaktion abgebrochen, ein Rollback ausgeführt und das Formular mit einem spezifischen Hinweistext neu gerendert.

GR-03 (Historiensicherheit):

Formale Bedingung:

$\forall x \in \{\text{Vitalwerte}, \text{Einnahmeprotokolle}\}: \text{UPDATE}(x) = \perp \wedge \text{DELETE}(x) = \perp$

Begründung:

Einmal erfasste Vitaldaten und Einnahmebestätigungen bilden die chronologische Dokumentationsgrundlage für die behandelnde Ärzteschaft. Das nachträgliche Manipulieren oder Stornieren historischer Einträge würde die Nachvollziehbarkeit der Einnahmetreue (Compliance) gefährden und den Wert des generierten Arztberichts entkräften.

Technische Durchsetzung:

- Architektonisch: Auf API- und Routing-Ebene werden für Protokolleinträge und historische Vitalwerte keine UPDATE- oder DELETE-Endpunkte für Standard-Anwender bereitgestellt.
- Datenbankebene: Neue Messungen werden ausschließlich über INSERT-Operationen als neue Datensätze angefügt.

GR-04 (Geburtsdatum):

Formale Bedingung:

geburtsdatum < *datum*_{aktuell}

Begründung:

Das Geburtsdatum der zu pflegenden Person dient als Stammdatenbasis für die Altersberechnung und Identifikation. Ein in der Zukunft liegendes Datum ist logisch ausgeschlossen.

Technische Durchsetzung:

- Clientseitig: Einschränkung des HTML5-Datepickers durch das dynamisch gesetzte Attribut `max="YYYY-MM-DD"`.
- Serverseitig: Vergleichende Prüfung gegen `datetime.date.today()` in der Python-Geschäftslogik vor dem Speichern der Patientendaten.

3.4 Schnittstellen & Validierung

Schnittstellen bilden die definierten Übergabepunkte eines Softwaresystems zu seiner Umwelt. Sie gliedern sich in Systemschnittstellen zur technischen Kommunikation zwischen Softwarekomponenten und Benutzerschnittstellen zur Human-Computer-Interaction. Die Schnittstellen stellen sicher, dass Daten sowohl technisch sicher verarbeitet werden als auch für zwei stark unterschiedliche Anwendergruppen ergonomisch zugänglich sind.

3.4.1 Systemschnittstellen

Die internen und externen Systemschnittstellen regeln den Datenaustausch zwischen den Komponenten der Anwendungsarchitektur. Sie stellen sicher, dass Informationen verlustfrei, typsicher und performant übertragen werden.

Internes HTTP-REST-Kommunikationsmodell (Frontend ↔ Backend)

Zweck: Übertragung von Benutzereingaben, Formularinhalten und Navigationsbefehlen zwischen dem Browser des Anwenders und dem Flask-Backend.

Protokoll & Methoden: Standardisiertes HTTP/1.1 via GET- und POST-Requests.

Datenformat: Formulardaten werden als `application/x-www-form-urlencoded` an den Server übermittelt. Bei dynamischen UI-Aktualisierungen (z. B. Statusänderungen in der Tagesansicht) werden serverseitig gerenderte HTML-Snippets oder strukturelle JSON-Antworten (`application/json`) ausgetauscht.

PDF-Reporting-Schnittstelle (Backend ↔ Export-Engine)

Zweck: Dynamische Erzeugung von aufbereiteten Arztberichten auf Basis historischer Datenbank-einträge.

Schnittstellenverhalten: Die Schnittstelle nimmt aggregierte Vital- und Medikationsdaten entgegen und verarbeitet diese über eine Template-Engine. Anschließend wird das generierte PDF-Dokument (application/pdf) per HTTP-Header (Content-Disposition: attachment) als Download bereitgestellt.

Persistenzschnittstelle (Backend ↔ Persistenzschicht)

Zweck: Abstraktion der persistenten Datenhaltung von der Domänenlogik zur Sicherung der ACID-Transaktionseigenschaften.

Schnittstellenverhalten: Entgegennahme typisierter Geschäftsobjekte aus der Fachlogik; Überführung in relationale Tabellenstrukturen; Sicherstellung referenzieller Integrität über Fremdschlüssel-Kaskadierungen sowie atomare Transaktionsabsicherung (Commit bei Erfolg, Rollback bei Ausnahmen).

3.4.2 Benutzerschnittstellen & UI-Konzept

Die Gestaltung der Benutzeroberfläche folgt einem dualen Schnittstellenkonzept, das den stark voneinander abweichenden Anforderungen der beiden primären Stakeholder-Gruppen Rechnung trägt.

Dashboard für pflegende Angehörige (Verwaltungsmodus)

Zielgruppe:

Pflegende Angehörige mit durchschnittlichen IT-Vorkenntnissen.

Struktur & Layout:

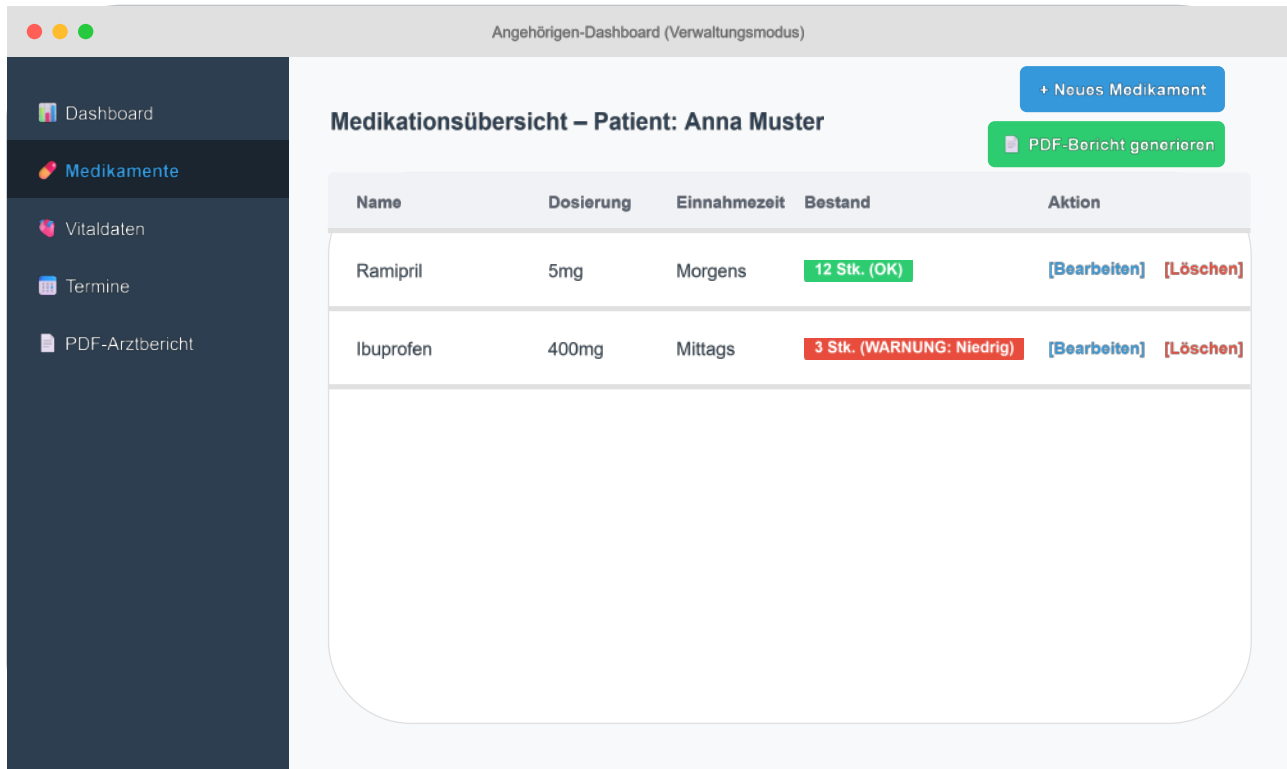
Die Oberfläche basiert auf einem responsiven Dashboard-Layout mit fixer Sidebar-Navigation. Das Design bietet eine hohe Informationsdichte und ermöglicht die effiziente Verwaltung aller Daten.

Interaktionselemente:

Tabellarische Ansichten zur Anzeige von Medikations- und Vitaldaten-Historien, Filter- und Suchfunktionen sowie modale Dialogfenster zur schnellen Erfassung neuer Datensätze, ohne die aktuelle Seite verlassen zu müssen.

Das grundlegende Verteilungsmuster und die funktionale Struktur des Verwaltungsdashboards werden in der nachfolgenden Skizze (Abbildung 4) dargestellt.

Abbildung 4 Verwaltungsdashboard



Quelle: eigene Darstellung

Das Mock-up verdeutlicht die klare Trennung von Hauptnavigation (links) und Arbeitsbereich (rechts). Durch den gezielten Einsatz von Tabellen-Aktionen und prominent platzierten Aktions-Buttons wird eine intuitive Bedienbarkeit bei administrativen Aufgaben gewährleistet.

Patientenmodus (Barrierefreie Tagesansicht)

Zielgruppe:

Zu pflegende Personen mit potenziell motorischen oder visuellen Einschränkungen und geringer IT-Affinität.

Struktur & Layout:

Kognitiv stark reduziertes Single-Page-Design. Verzicht auf störende Navigationsleisten, verschachtelte Menüs und Werbeelemente.

Interaktionselemente & Typografie:

Große, gut lesbare Sans-Serif-Schriftarten mit einer Basis-Schriftgröße von mindestens 18pt. Kontraststarke Farbgestaltung. Großflächige Touch-Buttons zur einfachen Bestätigung der Medikamenteneinnahme.

Die reduzierte und barrierefreie Benutzeroberfläche des Patientenmodus ist in Abbildung 5 illustriert.

Abbildung 5 Patientensicht

The screenshot shows a web application window titled "Patientensicht". On the left, it displays the date "Heute Donnerstag, 24. Juli". On the right, there are two medication reminder cards. The first card is for "08:00 Uhr Morgens" and "Vitamin D3", with a grey button that says "✓ Bereits eingenommen um 08:05 Uhr". The second card is for "12:00 Uhr - Mittags" and "Ramipril (5 mg)", with a green button that says "✓ Als eingenommen markieren".

Quelle: eigene Darstellung

Wie in Abbildung 5 erkennbar ist, konzentriert sich die Oberfläche ausschließlich auf die zentralen Fragen der zu pflegenden Person („Was muss ich jetzt einnehmen?“). Die Interaktion beschränkt sich auf das einfache Bestätigen über großflächige Schalter, was die Handlungsautonomie des Patienten fördert.

3.4.3 Mehrstufiges Validierungskonzept

Um Fehleingaben, unvollständige Formulare oder schadhafte Manipulationsversuche (z. B. Injection-Angriffe) wirksam abzufangen und die Systemstabilität zu garantieren, setzt die Anwendung auf ein mehrstufiges Validierungskonzept:

Clientseitige Validierung (HTML5 & Browser)

Bereits im Frontend werden Formulareingaben durch native HTML5-Attribute vorab geprüft. Attribute wie `required` (Pflichtfeld), `type="number"` (Erkennen von Falschformaten), `min="0"` (Verhinderung negativer Bestände) sowie `step="0.1"` bieten dem Nutzer eine unmittelbare Rückmeldung noch vor dem Absenden der Daten.

Serverseitige Validierung (Flask & Python)

Da clientseitige Prüfungen im Webbrowser umgangen werden können, führt das Flask-Backend bei jedem eingehenden Request eine erneute Prüfung aller Parameter durch. Datentypen werden explizit konvertiert (z. B. String zu Float) und fachliche Grenzwerte (gemäß den Geschäftsregeln aus Kapitel 3.3) evaluiert.

Fehlerbehandlung und Transaktionssicherheit

Schlägt die Validierung im Backend fehl, wird die angeforderte Datenbankoperation sofort abgebrochen und ein Rollback der Transaktion eingeleitet. Dem Anwender wird das Ursprungsformular mit einer spezifischen, gut verständlichen Fehlermeldung neu gerendert, sodass keine unvollständigen oder fehlerhaften Daten im System verbleiben.

4 Architekturdokumentation

Während die Kapitel 2 und 3 die konzeptionellen und fachlichen Anforderungen technologieunabhängig formalisiert haben, dokumentiert das vorliegende Kapitel die praktische Überführung in ein lauffähiges Softwaresystem. Hierzu werden die Technologieentscheidungen anhand der Qualitätsanforderungen kriteriengeleitet begründet, die modulare Drei-Schichten-Architektur dargelegt und die Klassen- sowie Interaktionsstrukturen im Quellcode formalisiert.

4.1 Technologieübersicht

Die Technologieauswahl basiert auf einer systematischen Kriterienanalyse. Tabelle 5 stellt die evaluierten Technologieoptionen anhand von Qualitätsmerkmalen vergleichend gegenüber.

Tabelle 5 Kriteriengeleitete Trade-Off-Analyse der Systemtechnologien

Architekturschicht	Evaluierte Optionen	Primäre Bewertungskriterien	Getroffene Entscheidung & Begründung
Backend-Framework	• Flask (Micro-Framework) • Django (Full-Stack) • FastAPI (Asynchron)	• Minimaler Footprint (NFA-05) • Geringe Angriffsfläche (NFA-03) • Lernkurve & Transparenz	Entscheidung für Flask: Django erzeugt durch Komponenten wie Admin-Schnittstelle, ORM-Zwang und Authentifizierungssystem einen monolithischen Overhead. Im privaten Single-User-Betrieb entstehen dadurch unnötige Sicherheitsrisiken. FastAPI fokussiert auf asynchrone APIs, bietet jedoch für serverseitig gerendertes HTML keinen Mehrwert gegenüber WSGI-Standards.
Persistenzschicht	• SQLite (Eingebettet) • PostgreSQL (Client-Server) • Flat-Files (JSON/CSV)	• Zero-Configuration (NFA-05) • ACID-Transaktionen (GR-01) • Wartungsfreiheit im Haushalt	Entscheidung für SQLite: PostgreSQL erfordert Hintergrunddienste und Port-Freigaben, was Laienanwender überfordert. JSON/CSV bieten keine Transaktionsisolation. SQLite garantiert volle ACID-Konformität in einer autarken Einzeldatei.
Frontend-Paradigma	• Server-Side-Rendering (SSR / Jinja2 + Bootstrap)	• Semantisches HTML / WCAG (NFA-01)	Entscheidung für SSR: SPAs verlagern Rendering-Last auf den Client

	• Single-Page-App (SPA / React/Vue + REST)	• Keine Node.js-Build-Pipeline • Geringe Latenz auf Altsystemen	und erfordern komplexe State-Synchronisationen. SSR via Jinja2 generiert deterministisches, standardkonformes HTML5, was die Durchsetzung von WCAG-AAA-Kontrasten und Großtypografie ohne clientseitige Script-Fehler absichert.
Dokumenten-Engine	• ReportLab (In-Memory) • Weasyprint (HTML-zu-PDF) • Headless Chrome	• Flüchtige Verarbeitung (Art. 9 DSGVO) • Deterministisches Drucklayout • Geringe Systemabhängigkeiten	Entscheidung für ReportLab: HTML-Konverter erfordern schwere externe C-Bibliotheken (Pango, Cairo) oder Browser-Binaries. ReportLab operiert nativ in Python und erlaubt über io.BytesIO die flüchtige In-Memory-Kompilierung ohne Festplatten-Spuren.

Quelle: eigene Darstellung

4.2 Architekturübersicht

Das architektonische Gesamtsystem folgt einer hierarchischen Drei-Schichten-Architektur, kombiniert mit dem klassischen MVC-Architekturmuster.

Schichten und Verantwortlichkeiten

Präsentationsschicht (View): Verantwortlich für das Rendering der Benutzeroberfläche und die Erfassung von Benutzerinteraktionen. Sie besteht aus serverseitig kompilierten Jinja2-Templates, die semantisches HTML5 generieren. Gemäß dem Prinzip der Separation of Concerns (SoC) sind Gestaltungsregeln und dynamisches Verhalten in isolierte Stylesheets (static/css/) und JavaScript-Dateien (static/js/) ausgelagert.

Steuerungs- und Geschäftslogikschicht (Controller / Service): Flask-Blueprints nehmen HTTP-Anfragen entgegen, steuern die Validierungskette über WTForms und führen die Geschäftsregeln aus. Komplexe Domänenlogiken sind in dedizierte Services ausgelagert (Service-Layer-Pattern). Diese Schicht garantiert, dass der Systemzustand konsistent bleibt und unberechtigte oder unplausible Eingaben abgefangen werden.

Datenzugriffs- und Persistenzschicht (Model): Definiert die Entitätsklassen und deren Beziehungen über das SQLAlchemy-ORM. Sie kapselt alle CRUD-Operationen gegen die relationale SQLite-Datenbank und erzwingt Constraints (z. B. Fremdschlüssel, Nicht-Null-Bedingungen).

Tabelle 6 MVC Zuordnungsmatrix

MVC-Komponente	Technologie	Projekt-Artefakte	Verantwortung
Model	Flask-SQLAlchemy, SQLite	app/models/*.py, database/medical_tracker.db	Persistierung der Geschäftsobjekte (Patient, Medikament, Vitalwert, Termin, EinnahmeProtokoll), Fremdschlüssel-Beziehungen und DB-Constraints (bestand >= 0).
View	Jinja2, Bootstrap 5, Vanilla JS	app/templates/**/*.html, app/static/css/, app/static/js/	Aufbereitung und visuelle Darstellung der UI-Zustände; barrierefreier Patientenmodus (WCAG 2.2 / NFA-01); dynamische Live-Uhr und Eingabemasken-Umschaltung.
Controller	Flask-Blueprints, Flask-WTF	app/routes/*.py, app/forms.py	Entgegennahme der HTTP-Anfragen, CSRF-Schutz, serverseitige Durchsetzung der Geschäftsregeln GR-01 bis GR-04, atomare Transaktionssteuerung und Weiterleitung (PRG-Pattern).
Service (Erweiterung)	ReportLab, Python io	app/services/pdf_generator.py	Entkoppelte Business-Komponente zur flüchtigen In-Memory-Kompilierung strukturierter DIN-A4-Arztberichte (FA-03 / NFA-03).

Quelle: eigene Darstellung

Zentrale Entwurfsmuster

Application-Factory-Muster (create_app()): Die Flask-Applikationsinstanz wird über eine Fabrikmethode dynamisch instanziiert.

Blueprint-Architektur: Zerlegung der Anwendung in funktionale Module (medikamente_bp, patient_bp, vitaldaten_bp etc.). Dies verhindert monolithische Controller-Dateien und stellt eine hohe Wartbarkeit sicher.

Service-Layer-Muster: Die Auslagerung der PDF-Erstellung in `app/services/pdf_generator.py` entkoppelt die Präsentations-Controller von der ReportLab-Grafikengine.

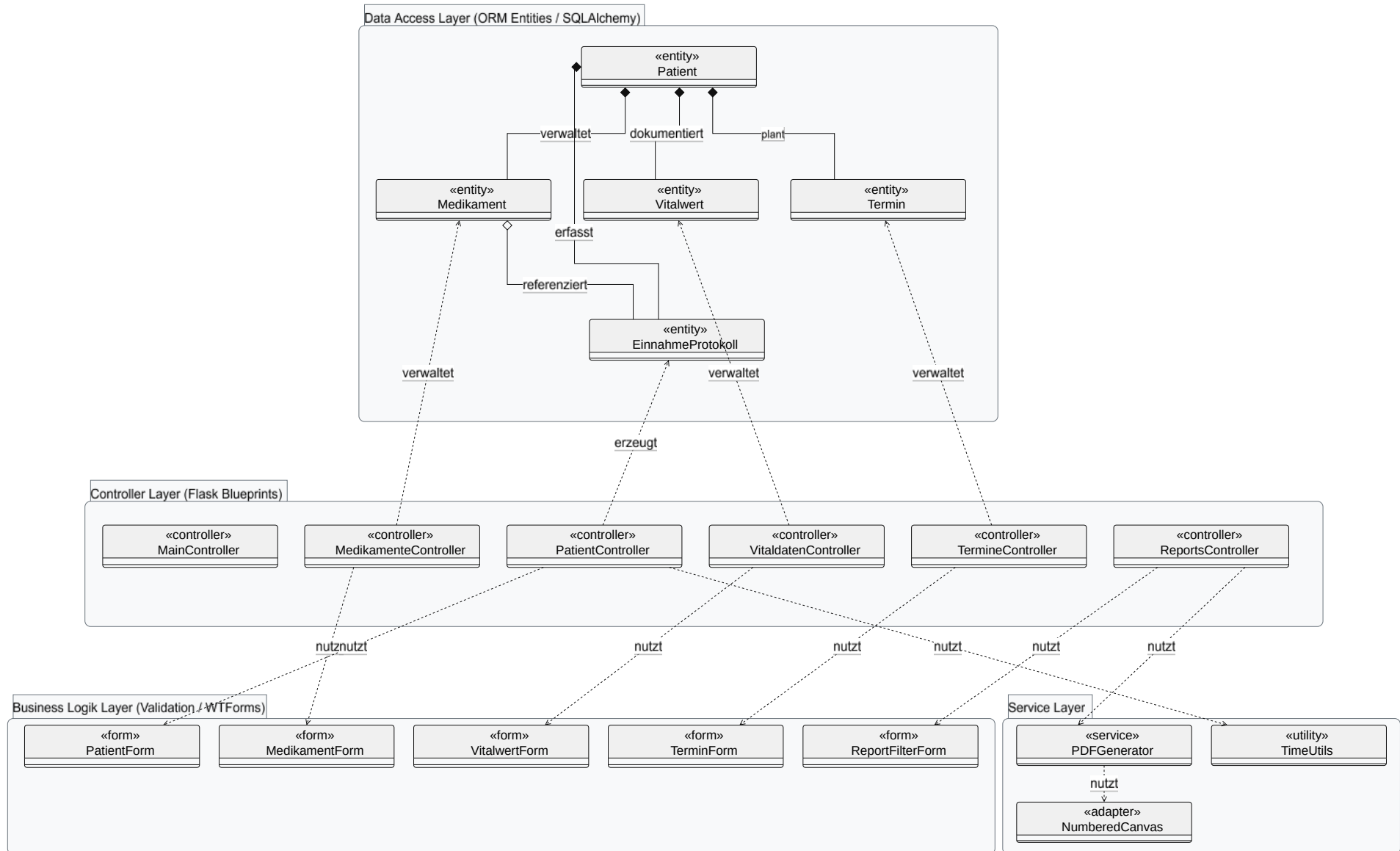
Zweistufiger Canvas-Renderer (NumberedCanvas): Ein strukturelles Adaptermuster überschreibt die `showPage-` und `save-`Methoden von ReportLab, um die exakte Gesamtseitenzahl zweistufig im Speicher zu akkumulieren und in der Kopf-/Fußzeile auszugeben.

4.3 Komponenten- und Klassenstruktur

Die statische Struktur der Anwendungssoftware gliedert sich in Datenmodelle, Validierungsformulare und funktionale Routencontroller. Auf der nachfolgenden Seite zeigt Abbildung 6 ein vereinfachtes Klassendiagramm. Eine vollständige Version ist im GitHub-Repository verfügbar.

<https://raw.githubusercontent.com/SteffenOe/medicaltracker/refs/heads/main/docs/diagramme/uml-klassendiagramm.drawio.svg>

Abbildung 6 Vereinfachtes Klassendiagramm des Gesamtsystems



Quelle: eigene Darstellung

Persistenzmodelle (Data Layer - app/models/)

Patient: Zentrales Wurzelobjekt des Datenmodells. Attribute: id (Integer, PK), vorname (String(100)), nachname (String(100)), geburtsdatum (Date), notfallkontakt (String(200)). Beziehungen: Verwaltet 1:n-Assoziationen zu Medikament, Vitalwert, Termin und EinnahmeProtokoll über Fremdschlüssel.

Medikament: Repräsentiert die verordnete Dauermedikation. Attribute: id (Integer, PK), patient_id (Integer, FK), name (String(150)), dosierung (String(50)), tageszeit (String(20)), bestand (Integer), mindestbestand (Integer).

EinnahmeProtokoll: Unveränderliches Dokumentationsobjekt (GR-03). Attribute: id (Integer, PK), patient_id (Integer, FK), medikament_id (Integer, FK), soll_tageszeit (String(50)), einnahme_zeitpunkt (DateTime), erledigt (Boolean).

Vitalwert: Zeitreihendokumentation physiologischer Messwerte. Attribute: id (Integer, PK), patient_id (Integer, FK), zeitpunkt (DateTime), kategorie (String(50)), wert (Float), einheit (String(20)).

Termin: Verwaltung von Pflegeterminen. Attribute: id (Integer, PK), patient_id (Integer, FK), titel (String(200)), kategorie (String(50)), zeitpunkt (DateTime), erledigt (Boolean).

Formular- und Validierungsklassen (Business Logik - app/forms/)

PatientForm: Validiert Eingaben für Personalien und erzwingt über die Methode validate_geburtsdatum() die Geschäftsregel GR-04 ($geburtsdatum < datum_{aktuell}$).

MedikamentForm: Prüft Pflichtangaben und stellt über NumberRange(min=0) sicher, dass Bestände und Schwellenwerte nicht negativ werden (GR-01).

VitalwertForm: Implementiert eine Validierungslogik in validate(): Erzwingt für Blutdruck das physiologische Intervall Blutdruck: $systolisch > diastolisch \wedge (30 \text{ mmHg} \leq diastolisch < systolisch \leq 300 \text{ mmHg})$, für die Pulsfrequenz: $30 \text{ bpm} \leq f_{puls} \leq 250 \text{ bpm}$ und für das Körpergewicht: $2,0 \text{ kg} \leq m \leq 300,0 \text{ kg}$ (GR-02).

TerminForm: Erfasst über DateTimeLocalField strukturierte Datums- und Uhrzeitwerte für den Terminkalender.

ReportFilterForm: Steuert die Zeitfensterabgrenzung für den PDF-Export und prüft chronologische Konsistenz ($start_datum < end_datum$).

4.4 Dynamisches Systemverhalten (Verhaltenssicht – Kernprozess GP-03)

Der Kernprozess GP-03 (Medikamenteneinnahme und Bestandsaktualisierung) verknüpft die Interaktion im barrierefreien Patientenmodus mit der transaktionssicheren Fortschreibung des Lagerbestands und einer revisionssicheren Protokollierung. Um die softwarearchitektonischen Aspekte übersichtlich darzustellen, wird der Gesamtablauf nachfolgend in zwei Phasen unterteilt: die initiale Datenaggregation für die Tagesansicht sowie die transaktionssichere Einnahmeverbuchung.

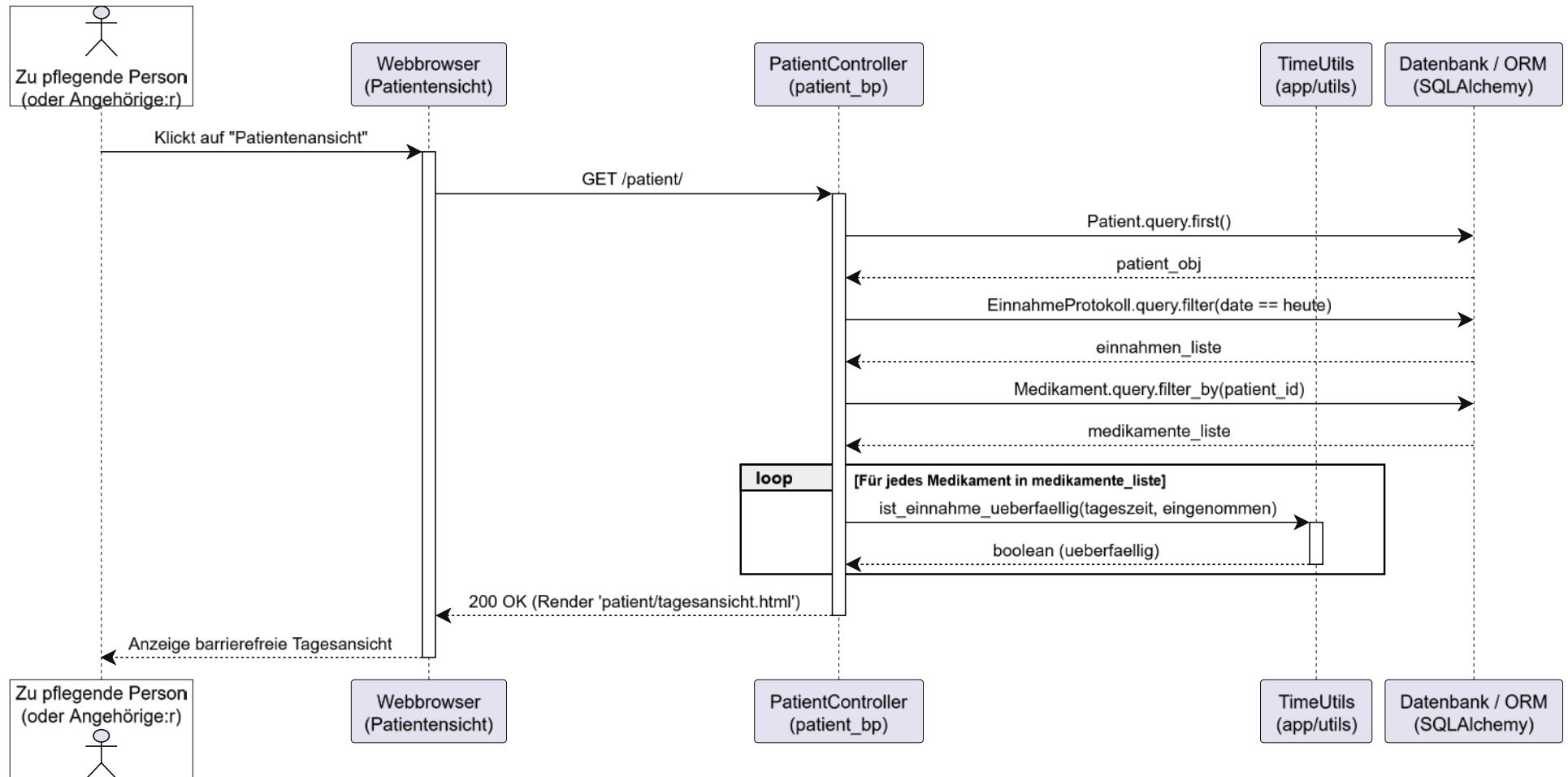
Initiale Datenaufbereitung und Bereitstellung der Tagesansicht

Das Laden der Benutzeroberfläche umfasst das Auslesen des aktuellen Patientenkontexts sowie die dynamische Statusprüfung der anstehenden Einnahmen (siehe Abbildung 7).

Der Ablauf gliedert sich in folgende Schritte:

1. Aufruf der Tagesansicht: Der Client sendet einen GET /patient/-Request an das Backend. Der Controller (patient_bp) fragt über die Abstraktionsschicht des ORM (SQLAlchemy) die Stammdaten des Patienten sowie die für den aktuellen Tag bereits existierenden Einträge der Entität EinnahmeProtokoll ab.
2. Datenaggregation & Zeitprüfung: Das System lädt die verordneten Medikamente. In einer Schleife iteriert der Controller über diese Kollektion und ermittelt über das Hilfsmodul TimeUtils (app/utils.py) anhand der aktuellen Systemzeit, ob eine Einnahme fällig oder überfällig ist.
3. Auslieferung der View: Die konsolidierten Daten werden an die Template-Engine übergeben, welche die Datei patient/tagesansicht.html mit dem HTTP-Statuscode 200 OK rendert und im Webbrowser als barrierefreie Benutzeroberfläche anzeigt.

Abbildung 7 UML-Sequenzdiagramm zur Bereitstellung der barrierefreien Tagesansicht



Quelle: eigene Darstellung

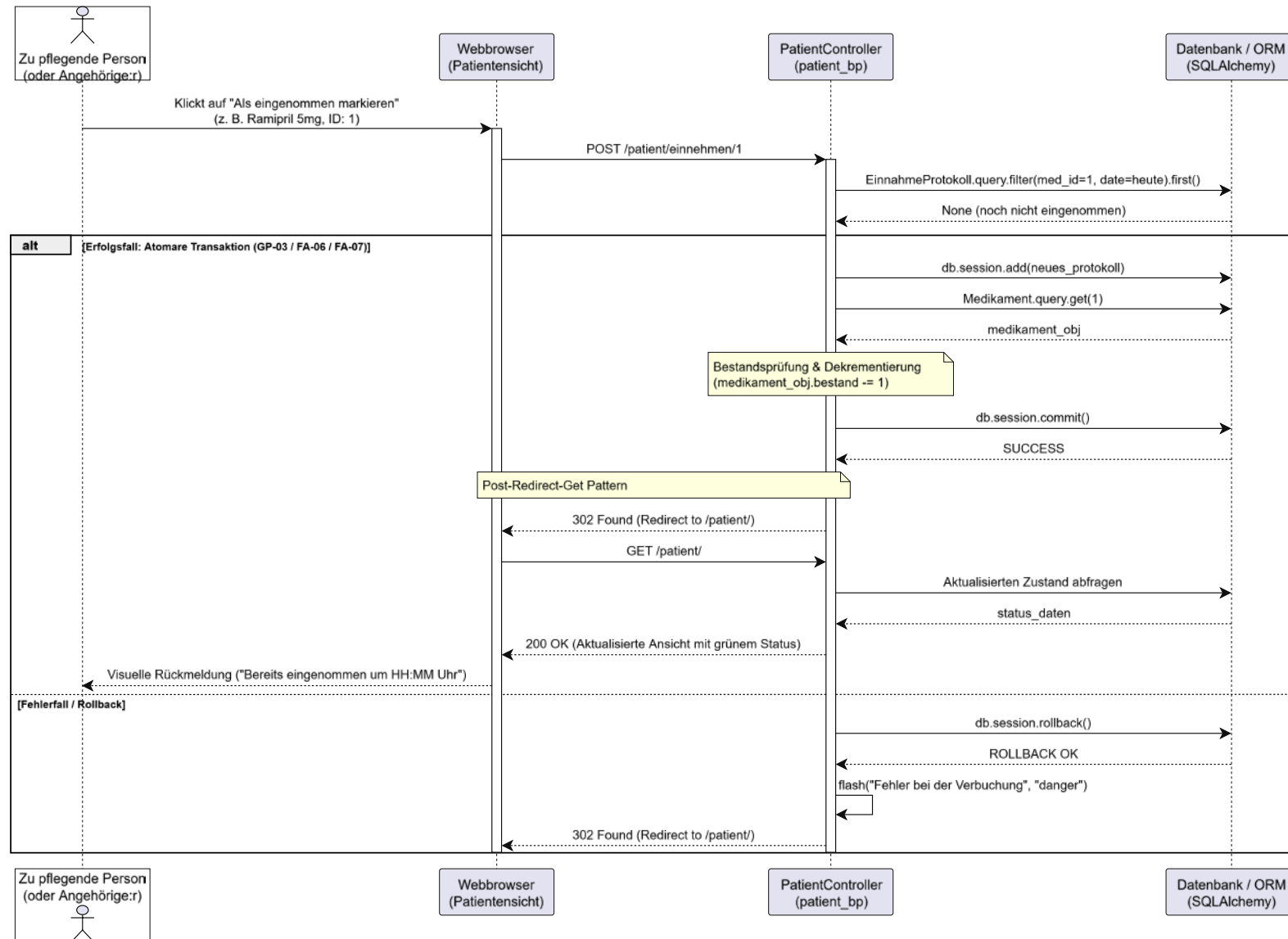
Transaktionssichere Einnahmeverbuchung

Die eigentliche Verbuchung einer Einnahme erfordert strikte Konsistenzprüfungen und Transaktionssicherheit (ACID), um Fehl- und Doppelbuchungen im Datenbestand auszuschließen (siehe Abbildung 8).

Der Ablauf vollzieht sich wie folgt:

1. Auslösen der Einnahmebestätigung: Die anwendende Person betätigt die Schaltfläche „Als eingenommen markieren“. Der Browser übermittelt daraufhin einen POST /patient/einnehmen/<id>-Request an das Backend.
2. Duplikatsprüfung: Vor Beginn der Transaktion prüft der Controller über das ORM, ob für die Kombination aus medikament_id und aktuellem Tagesdatum bereits ein Datensatz vorliegt. Dies verhindert Mehrfachbuchungen bei wiederholter Betätigung.
3. Atomare Verbuchung (alt-Zweig: Erfolgsfall): Liegt noch keine Bestätigung vor, instanziiert das System ein neues EinnahmeProtokoll-Objekt (datetime.now()) und registriert dieses in der Session (db.session.add()). Parallel wird der referenzierte Datensatz aus Medikament geladen und dessen Bestand dekrementiert (bestand = bestand - 1, Geschäftsregel GR-01). Mittels db.session.commit() werden beide Operationen atomar in der Datenbank persistiert.
4. Fehlerbehandlung (alt-Zweig: Rollback): Tritt während des Schreibvorgangs eine Exception auf (z. B. Sperrkonflikt oder Integritätsverletzung), wird die Transaktion via db.session.rollback() vollständig zurückgerollt, sodass kein inkonsistenter Zustand verbleibt. Der Controller erzeugt eine entsprechende Flash-Fehlermeldung.
5. Post-Redirect-Get-Muster (PRG): Das Post-Redirect-Get-Muster verhindert Mehrfachübermittlungen beim Neuladen der Seite: Nach dem POST-Request antwortet der Server mit einem 302 Found-Redirect auf /patient/. Der Browser setzt daraufhin einen idempotenten GET-Request ab und rendert die quitierte Dosis als deaktivierte, grüne Kachel.

Abbildung 8 UML-Sequenzdiagramm zur Medikamenteneinnahme



Quelle: eigene Darstellung

5 Testdokumentation

Die Qualitätssicherung dient der systematischen und empirischen Verifikation der in Kapitel 2 erhobenen funktionalen und nichtfunktionalen Anforderungen sowie der in Kapitel 3 formalisierten Geschäftsregeln und Schnittstellenkontrakte. Der Qualitätssicherungsprozess gliedert sich in eine strukturierte Teststrategie, die Festlegung von Testumgebung und -werkzeugen sowie ein tabellarisches Testprotokoll

5.1 Teststrategie

Die Qualitätssicherung des Softwaresystems stützt sich methodisch auf die Testpyramide nach Mike Cohn sowie die normativen Vorgaben des Standards ISO/IEC/IEEE 29119 für Softwaretests. Das Qualitätssicherungskonzept weist die funktionale Korrektheit und die Einhaltung der Qualitätsmetriken empirisch nach. Hierzu nutzt es drei hierarchische Teststufen: atomare Modultests, schichtenübergreifende Integrationstests sowie ganzheitliche System- und Akzeptanzprüfungen.

Auf der untersten Ebene der Testpyramide fokussieren die Modultests auf die isolierte Verifikation atomarer Logikbausteine ohne Abhängigkeiten zu externen Laufzeitkomponenten wie Netzwerkprotokollen oder dem physischen Dateisystem. Im Mittelpunkt dieser Teststufe steht die Verifikation der serverseitigen Validierungslogik innerhalb der Formularmodule, welche die Einhaltung der fachlichen Invarianten gewährleisten. Hierunter fällt die Geschäftsregel zur Bestandspositivität (GR-01). Die Modultests stellen sicher, dass Arzneimittelbestände keine negativen Werte annehmen können. Ein Nullbestand wird dabei als legitimer Grenzwert verarbeitet. Ebenso validieren die Modultests die physiologischen Schwellenwertkorridore für Blutdruck, Pulsfrequenz und Körpergewicht, indem paradoxe Druckverhältnisse und unplausible Extremwerte vor Erreichen der Persistenzschicht deterministisch abgewiesen werden. Ergänzend sichern Modultests die relationale Integrität der Domänenmodelle ab, überprüfen kaskadierende Löschregeln auf Entitätsebene und verifizieren über isolierte Testroutinen die logische Korrektheit der zeitlichen Überfälligkeitsberechnungen für anstehende Einnahmezeitfenster.

Die mittlere Teststufe bilden Integrationstests, welche die funktionale Interaktion und den Datenfluss über die Schnittstellen von Präsentations-, Steuerungs- und Persistenzschicht hinweg überprüfen. Zentraler Gegenstand dieser Ebene ist die End-to-End-Abbildung des Kernprozesses der Medikamenteneinnahme. Hierbei wird das synchrone Zusammenspiel zwischen der Generierung eines unveränderlichen Protokolldatensatzes, der Dekrementierung des Lagerbestands und der Schwellenwertauswertung für optische Nachbestellwarnungen verifiziert. Zugleich prüfen die Integrationstests die Durchsetzung des Post-Redirect-Get-Musters zur Vermeidung versehentlicher Doppelbuchungen bei wiederholten Browseranfragen. Neben dem Kernprozess verifizieren Integrationstests die vollständigen Lebenszyklen aller funktionalen Routen-Blueprints für Medikamente, Vitalwerte, Termine und Patientenstammdaten sowie die entkoppelte Ausführung der Reporting-Komponente. Bei

letzterer wird die flüchtige In-Memory-Kompilierung von PDF-Dokumenten über ReportLab und deren fehlerfreie Übertragung als HTTP-Bytestrom ohne temporäre Dateisystemrückstände nachgewiesen.

Die oberste Teststufe umfasst System- und Akzeptanztests, welche das Gesamtsystem unter realitätsnahen Betriebsbedingungen evaluieren. Diese Tests widmen sich vorrangig der Mensch-Maschine-Schnittstelle und den nichtfunktionalen Produktmerkmalen. Im Fokus steht die empirische Überprüfung der Barrierefreiheitskriterien im Patientenmodus, bei der die berechneten Stile des gerenderten Document Object Models hinsichtlich der vorgeschriebenen Basis-Schriftgrößen von mindestens 18 pt, großflächiger Interaktionselemente sowie der Kontrastverhältnisse gemäß den WCAG überprüft werden. Parallel dazu evaluieren Systemtests die softwarearchitektonische Robustheit gegenüber sicherheitskritischen Eingabemustern, indem die Immunität der Schnittstellen gegen Script-Injektionen und relationale Injektionsversuche im Zusammenspiel von Template-Engine und Objekt-Relationalem Mapping überprüft wird.

5.2 Testumgebung

Die Testinfrastruktur ist vollständig in die automatisierte Entwicklungsumgebung integriert und nutzt standardisierte Software-Engineering-Werkzeuge:

- Test-Framework: pytest (Version 9.1.1) mit dem Coverage-Plugin pytest-cov (Version 7.1.0) unter Python 3.14.
- Datenbankisolation: Zur Vermeidung von Wechselwirkungen zwischen einzelnen Testläufen und zum Schutz produktiver Datensätze, wird eine flüchtige In-Memory-SQLite-Datenbank instanziiert. Nach jedem Testfall wird die Datenbankstruktur bereinigt.
- HTTP-Test-Client: Die Ansteuerung der Routen und Controller erfolgt über den zustandslosen Test-Client von Flask. Der CSRF-Schutz wird für automatisierte Testläufe deaktiviert, um eine programmatische POST-Formularübermittlung zu ermöglichen.
- Konfiguration: Die Steuerung der Testsuite erfolgt über die zentrale Konfigurationsdatei `pytest.ini` im Projektstammverzeichnis

5.3 Formales Testprotokoll

Tabelle 7 dokumentiert die 17 automatisierten Testfälle und stellt die Verifikation der funktionalen Anforderungen FA-01 bis FA-07, der Geschäftsregeln GR-01 bis GR-04 sowie der nichtfunktionalen Qualitätsmerkmale dar.

Tabelle 7 Formales Testprotokoll der automatisierten Testsuite

Testfall-ID	Kurzbeschreibung & Ziel	Vorbedingung	Eingabedaten / Benutzeraktion	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC-FA-01-01	Medikament erfassen (FA-01)	Patient existiert in Datenbank	POST /medikamente/neu mit Name: 'Amlodipin 5mg', Dosis: '1 Tablette', Zeit: 'Abends', Bestand: 28, Min: 7	HTTP 302 Redirect auf /medikamente/; Datensatz persistent in SQLite gespeichert; Bestand = 28	HTTP 302 Redirect; Datensatz in DB persistent angelegt; Anzeige in Medikationsplan synchron	PASSED
TC-FA-01-02	Medikament modifizieren & löschen (FA-01)	Medikament 'Amlodipin 5mg' existiert	1. POST /medikamente/<id>/bearbeiten (Dosis auf '10mg'), 2. POST /medikamente/<id>/loeschen	Attributänderung persistent übernommen; nach Löschung ist Datensatz vollständig entfernt	Name erfolgreich auf '10mg' aktualisiert; nach Löschung liefert session.get() den Wert None	PASSED
TC-GR-01-01	Negativen Bestand abfangen (GR-01)	MedikamentForm im Request-Kontext instanziiert	Formulardaten mit bestand = -1 und mindestbestand = -5 übermitteln	Client- und serverseitige Validierung schlägt fehl; Formular meldet Bereichsfehler	validate() == False; Fehlermeldungen in form.errors['bestand'] registriert	PASSED
TC-GR-01-02	Grenzwertprüfung Bestand = 0 (GR-01)	MedikamentForm instanziiert	Formulardaten mit bestand = 0 und mindestbestand = 0 absenden	Formularvalidierung ist erfolgreich (InputRequired akzeptiert numerische Null)	validate() == True; Nullbestand wird als legitimer Grenzwert akzeptiert	PASSED
TC-FA-02-01	Vitalwert-Tracking erfassen (FA-02)	Patient in Datenbank vorhanden	POST /vitaldaten/neu für Blutdruck (128/84 mmHg), Puls (74 bpm) und Gewicht (78,2 kg)	HTTP 302 Redirect; Speicherung mit aktuellem Server-Zeitstempel in der Verlaufshistorie	Alle drei Messarten persistent in SQLite angelegt; chronologische Sortierung gewahrt	PASSED
TC-GR-02-01	Blutdruck-Plausibilität erzwingen (GR-02)	VitalwertForm geöffnet	1. Systolisch: 80, Diastolisch: 120 (Sys <= Dia), 2. Systolisch: 320 (> 300 mmHg)	Formular weist unplausible Werte ab; Datenbanktransaktion wird blockiert	validate() == False; Plausibilitätsfehler für systolisch gemeldet	PASSED
TC-GR-02-02	Schwellenwerte Puls & Gewicht (GR-02)	VitalwertForm geöffnet	1. Puls: 25 bpm (< 30), 2. Gewicht: 350 kg (> 300)	Grenzverletzungen werden abgewiesen; Normalwerte (72 bpm, 75,5 kg) werden akzeptiert	Fehleingaben blockiert; Normalwerte fehlerfrei validiert (True)	PASSED
TC-FA-03-01	In-Memory-PDF-Generierung (FA-03, NFA-03)	Patient, Medikation und Historie vorhanden	Aufruf von erstelle_arztbericht_pdf() mit 30-Tage-Filter	Rückgabe eines flüchtigen io.BytesIO-Bytestroms; Beginn mit Signatur %PDF-	Valides PDF im RAM erzeugt; 0 Byte temporäre Zwischendateien im Dateisystem	PASSED
TC-FA-03-02	Berichts-Download-Route (FA-03)	Patient vorhanden	POST /berichte/ mit gültigem Datumsbereich absenden	HTTP 200; Content-Type: application/pdf; Header Content-Disposition: attachment	PDF-Download fehlerfrei ausgeliefert; Dateiname enthält Patientenname und Zeitraum	PASSED
TC-FA-03-03	Datumsbereichs-Konsistenz (FA-03)	BerichtForm instanziiert	Startdatum chronologisch nach dem Enddatum übergeben (start > end)	Formularvalidierung schlägt fehl; chronologische Inkonsistenz abgewiesen	validate() == False; Fehlermeldung an start_datum gerendert	PASSED
TC-FA-04-01	Pflegetermin-Lifecycle & Toggle (FA-04)	Patient vorhanden	POST /termine/neu, anschließend POST /termine/<id>/toggle	Termin angelegt (erledigt=False); wechselt nach Toggle auf erledigt=True	Status in SQLite aktualisiert; visuelle Kennzeichnung im Board angepasst	PASSED

TC-FA-05-01	Patientenmodus & Profilverwaltung (FA-05)	Patient vorhanden	1. GET /patient/, 2. POST /patient/profil (Stammdaten-aktualisierung)	Tagesansicht liefert HTTP 200; Stammdatenänderung persistent gespeichert	Patientenansicht ohne Menübelast dargestellt; Profil persistent modifiziert	PASSED
TC-FA-06-01	Kernprozess GP-03: Einnahme & Bestand (FA-06, FA-07)	Medikament mit Bestand = 10 vorhanden	Klick auf 'Als eingenommen markieren' (POST /patient/einnehmen/<id>)	EinnahmeProtokoll erstellt; Bestand dekrementiert auf 9; HTTP 302 Redirect	Protokollsatz persistent; Bestand synchron dekrementiert; Ansicht aktualisiert	PASSED
TC-FA-07-01	Schwellenwert-Warnung bei Mindestbestand (FA-07)	Medikament mit Bestand = 4, Mindestbestand = 3	Einnahme ausführen (bestand sinkt auf 3 -> Schwellenwert erreicht)	Dashboard rendert Nachbestell-Warnung ('Achtung: Nachbestellung erforderlich!')	bestand == 3; Warnbox mit Badge bg-danger im Dashboard gerendert	PASSED
TC-GR-03-01	Duplikats- & Historien-schutz (GR-03)	Medikament für heute bereits quittiert	Erneutes Absenden von POST /patient/einnehmen/<id> am selben Kalendertag	Zweite Einnahme wird ignoriert; Bestand wird nicht erneut dekrementiert	Exakt ein Protokolleintrag bleibt bestehen; Bestand unverändert geschützt	PASSED
TC-GR-04-01	Geburtsdatum-Validierung (GR-04)	PatientForm instanziiert	Geburtsdatum auf morgigen Kalendertag setzen	form.validate() schlägt fehl (geburtsdatum < datum_aktuell verletzt)	ValidationError ausgelöst; Speicherung ungeborener Personen blockiert	PASSED
TC-UT-01-01	Logische Überfälligkeitsbewertung	Hilfsmodul ueberfaellig.py	Evaluation aller Zeitslots (Morgens, Mittags, Abends, Nachts, Bei Bedarf)	Exakte boolesche Rückgabe; Bedarfsmedikation wird niemals überfällig	Logische Zweige zu 100 % überdeckt; deterministisches Zeitverhalten	PASSED

Quelle: eigene Darstellung

5.4 Analyse der Testabdeckung

Zur quantitativen Überprüfung der Anforderung NFA-06 (Wartbarkeit & Testbarkeit) wurde die Testabdeckung über das Modul `pytest-cov` gemessen. Tabelle 8 stellt die Statement- und Coverage-Verteilung über die Schichten der Anwendungsarchitektur dar.

Tabelle 8 Analyse der Testabdeckung

Modul / Komponente	Statements (LOC)	Nicht abgedeckt	Abgedeckt	Ziel (NFA-06)
app/__init__.py (Application Factory)	20	0	100%	>= 80 %
app/models/ (ORM-Entitäten gesamt)	72	5	93%	>= 80 %
app/forms/ (Formulare & Validierungslogik)	127	11	91%	>= 80 %
app/routes/ (Flask-Blueprints & Controller)	256	22	91%	>= 80 %
app/services/pdf_generator.py (Export-Engine)	87	13	85%	>= 80 %
app/utils/ (Hilfsfunktionen & Zeitlogik)	12	0	100%	>= 80 %
Gesamtsystem	568	58	90%	>= 80 %

Quelle: eigene Darstellung

Mit einer Gesamtabdeckung von 90 % übertrifft das Softwaresystem den in NFA-06 definierten Mindestschwellenwert von 80 % deutlich. Die verbleibenden 58 nicht abgedeckten Zeilen entfallen auf defensive Ausnahmebehandlungen bei hypothetischen Systemfehlern (z. B. fehlschlagende Betriebssystem-Dateizugriffe oder Verbindungsabbrüche während des Renderns) sowie auf die interaktive Starterroutine.

5.5 Verifikation der nichtfunktionalen Anforderungen

Die Verifikation der Qualitätsanforderungen nach ISO/IEC 25010 erfolgte durch automatisierte Lasttests und messtechnische Inspektionen der Benutzeroberfläche

5.5.1 Performanz- und Skalierbarkeitsbenchmark

Zur empirischen Absicherung der Skalierbarkeit über mehrjährige Dokumentationszeiträume wurde ein automatisierter Lasttest (`tests/integration/test_performance.py`) durchgeführt. Hierbei wurde die SQLite-Datenbank über einen Bulk-Insert mit 10.000 persistenten Datensätzen (5.000 Vitalwerte und 5.000 Einnahmeprotokolle) befüllt, was einem mehrjährigen Pflegeverlauf entspricht. Die Zeitmessung erfolgte über `time.perf_counter()`.

- **Massendaten-Persistierung:** Der Schreibvorgang für 10.000 relationale Datensätze benötigte 0,253 s (Durchsatz: ~39.500 Datensätze/s).
- **Datenbank-Latenz (NFA-02 / NFA-07):** Die chronologische Abfrage der fünf neuesten Vitalwerte aus dem Datenbestand von 10.000 Datensätzen beanspruchte 1,65 ms. Damit wird die geforderte Maximalgrenze von 200 ms um mehr als den Faktor 120 unterschritten. Die Indizierung über Fremdschlüssel und Primärschlüssel verhindert lineare Tabellenscans.
- **View-Ladezeit Dashboard (NFA-02):** Das vollständige serverseitige Rendern des Dashboards unter Einbeziehung von Bestandsprüfungen und Terminen dauerte 0,033 s (33 ms). Die Anforderung von maximal 1,0 s wird mit einer Reserve von über 96 % eingehalten.
- **In-Memory-PDF-Kompilierung (NFA-02):** Die Erstellung eines strukturierten DIN-A4-Arztberichts mit 734 historischen Messwerten über ReportLab benötigte 1,234 s. Das geforderte Zeitlimit von maximal 2,0 s wird somit auch bei umfangreichen Historien sicher eingehalten.

5.5.2 Barrierefreiheit und Ergonomie

Die Einhaltung der Barrierefreiheitskriterien für geriatrische Zielgruppen wurde im isolierten Patientenmodus messtechnisch verifiziert.

- **Farbkontraste:** Die kolorimetrische Auswertung nach der W3C-Luminanzformel ergibt für handlungsleitende Textelemente (#212529 auf #f8f9fa) ein Kontrastverhältnis von 14,2:1 sowie für die Erfolgs-Bestätigungskacheln (ffffff auf #198754 bzw. #1b5e20 auf #e8f5e9) ein Verhältnis von 7,8:1. Beide Werte übertreffen die WCAG-2.2-Stufe AAA (Mindestanforderung 7,0:1).
- **Typografie:** Gemäß der W3C-Referenzumrechnung (1 pt = 1,333 px) entspricht die geforderte Mindestgröße von 18 pt exakt 24 px. Die Benutzeroberfläche des Patientenmodus setzt dies über eine Basisschriftgröße von 1.5 rem (24 px) im Body-Element durch.
- **Trefferflächen:** Die Bestätigungsschaltflächen weisen eine Höhe von mindestens 54 px und eine Breite von 100 % des Lesebereichs (780 px) auf. Das Kriterium von 48 x 48 px ist somit erfüllt.

6 Limitationen, Risiken & Technische Schulden

Das vorliegende Kapitel unterzieht das entwickelte Softwaresystem einer kritischen softwaretechnischen Bestandsaufnahme und Reflexion. Auf Basis der vorangegangenen Implementierung und empirischen Qualitätssicherung werden zunächst die funktionalen und architektonischen Limitationen offengelegt, die sich aus dem abgegrenzten Zuschnitt des MVP für den privaten Haushaltseinsatz ergeben. Darauf aufbauend werden die im Entwicklungsprozess eingegangenen technischen Schulden transparent dokumentiert, hinsichtlich ihrer systemspezifischen Risiken analysiert und mit konkreten Refactoring-Maßnahmen für Folgeiterationen versehen. Den Abschluss bildet eine retrospektive Risikobewertung, welche die zu Projektbeginn definierten Projektrisiken mit den tatsächlichen Resultaten der Umsetzung abgleicht und die Wirksamkeit der getroffenen Vermeidungs- und Schutzstrategien evaluiert.

6.1 Technische und funktionale Limitationen

Das entwickelte Softwaresystem fokussiert im Rahmen des MVP auf die Entlastung im privaten, häuslichen Nutzungskontext. Aus dieser bewussten Zielgruppendefinition resultieren technische und funktionale Restriktionen, die das System gegenüber enterprise-orientierten Pflegesystemen abgrenzen.

Die Anwendung verzichtet im aktuellen Entwicklungsstand auf eine softwarebasierte Benutzerauthentifizierung und ein rollenbasiertes Zugriffskontrollsystem (Role-Based Access Control, RBAC). Die funktionale Trennung zwischen dem administrativen Angehörigen-Dashboard und der kognitiv reduzierten Patientensicht wird auf Anwendungsebene rein über logische URL-Pfade geregelt. In einem geteilten Nutzungsszenario im Haushalt, beispielsweise bei Betrieb auf einem gemeinsamen Familien-PC, existiert somit keine softwareseitige Barriere, die eine versehentliche Konfigurationsänderung durch die pflegebedürftige Person unterbindet. Die Vertraulichkeit und Zugriffintegrität ist infolgedessen an organisatorische Maßnahmen sowie an die Benutzerkontentrennung des zugrunde liegenden Betriebssystems delegiert.

Darüber hinaus beruht die Präsentationsschicht auf dem klassischen Paradigma des Server-Side-Renderings über Jinja2-Templates und synchrone HTTP-Request-Response-Zyklen. Quittiert die zu pflegende Person eine Einnahme am Tablet, schreibt das System Protokoll und Lagerbestand zwar atomar fort. Das Dashboard auf einem Zweitgerät aktualisiert seinen Status jedoch nicht in Echtzeit, da eine asynchrone Push-Kommunikation fehlt. Das Fehlen einer asynchronen Push-Kommunikation (z. B. via WebSockets oder Server-Sent Events) bedingt, dass Bestandsänderungen erst nach einer manuellen Seitenaktualisierung durch den Angehörigen sichtbar werden.

Hinsichtlich der Datensicherheit im Ruhezustand (Data-at-Rest) verbleibt die Persistenzdatei unverschlüsselt im lokalen Dateisystem. Zwar werden Schutzmechanismen gegen SQL-Injections durch das objektrelationale Mapping via SQLAlchemy und gegen Cross-Site-Scripting durch Jinja2-Auto-

Escaping durchgesetzt, gegen physischen Datendiebstahl bietet die SQLite-Datenbankdatei jedoch keinen kryptographischen Eigenschutz. Der Schutz der sensiblen Gesundheitsdaten gemäß Art. 9 DSGVO erfordert daher zwingend flankierende Festplattenverschlüsselungsverfahren des Host-Betriebssystems (z. B. Microsoft BitLocker).

6.2 Erfassung technischer Schulden

Technische Schulden beschreiben das bewusste Eingehen kurzfristiger Kompromisse bei Architektur oder Implementierung, um funktionale Kernziele termingerecht zu erreichen. Tabelle 9 erfasst die im System verankerten technischen Schulden, bewertet deren Risikopotenzial und definiert konkrete Kompensationsmaßnahmen für Folgeversionen:

Tabelle 9 Erfassung technischer Schulden

Schuld-ID & Kategorie	Beschreibung der technischen Schuld	Ursache und Rationale im Projekt	Technisches Risiko & Auswirkung	Geplante Kompensationsmaßnahme (Refactoring)
TD-01: Sicherheit & Berechtigungen	Fehlen eines Authentifizierungsmoduls, Session-Handlings und RBAC-Rollenschemas	Vermeidung von Einstiegshürden; Gewährleistung eines barrierefreien Zero-Configuration-Deployments für Laien	Unbefugter Zugriff auf Medikations- und Vitaldaten im lokalen Netzwerk bei Freigabe von Host-Port 5000	Kapselung der Blueprints mit Flask-Login und Einführung einer Berechtigungsmatrix (Admin, Angehöriger, Patient)
TD-02: Architektur & Synchronisation	Verzicht auf bidirektionale Echtzeit-Events (keine WebSockets / SSE)	Vermeidung asynchroner Protokollkomplexität und komplexer JavaScript-Framework-Pipelines	Latenzbehafteter Informationsversatz zwischen Einnahmequittierung am Tablet und Angehörigen-Dashboard	Anbindung einer leichtgewichtigen Server-Sent-Events-Route (SSE) zur automatischen Triggern von DOM-Updates
TD-03: Persistenz & Concurrency	SQLite nutzt dateiweites Table-Locking bei transaktionalen Schreiboperationen	Autarke, installationsfreie Datenhaltung als dateibasierte Einzeldatei ohne separaten RDBMS-Dienst	Potenzielle Schreibblockaden (OperationalError: database is locked) bei parallelem Mehrgerätebetrieb	Nutzung der SQLAlchemy-Abstraktion zur Konfiguration einer Client-Server-Datenbank (PostgreSQL nach NFA-07)
TD-04: Kryptographie	Unverschlüsselte Persistenz der SQLite-Datei im Anwendungsverzeichnis	Vermeidung schwergewichtiger nativer C-Bibliotheksabhängigkeiten zur Erhaltung der Windows-Portabilität	Verletzung der Vertraulichkeit nach Art. 9 DSGVO bei Verlust unverschlüsselter Notebook-Hardware	Einbindung der Treibererweiterung SQLCipher zur transparenten 256-Bit-AES-Vollverschlüsselung der Datenbankdatei

Quelle: eigene Darstellung

6.3 Retrospektive Risikobewertung

Die systematische Risikoanalyse zu Projektbeginn (Abschnitt 1.6, Tabelle 1) definierte drei primäre Projektrisiken. Der retrospektive Abgleich nach Abschluss der Implementierungs- und Testphase belegt die Wirksamkeit der implementierten Mitigationen.

Das technische Risiko eines Datenverlusts durch Systemabstürze (R-01) wurde durch den Einsatz von SQLAlchemy-Transaktionen mit automatischem Rollback im Fehlerfall sowie durch die integrierte Startroutine eliminiert. Vor jedem Systemstart erfolgt eine automatisierte SQLite-Header-Integritätsprüfung (PRAGMA integrity_check) und die Erstellung einer zeitgestempelten Sicherungskopie, wodurch ein Datenverlust wirksam verhindert wird.

Das ergonomische Design-Risiko einer unzureichenden Usability für geriatrische Anwender (R-02) wurde durch das duale Schnittstellenkonzept erfolgreich abgewendet. Durch die physische Entkopplung des administrativen Dashboards von der barrierefreien Tagesansicht, die Implementierung großflächiger Touch-Elemente sowie die WCAG-2.2-AAA-konformen Farbkontraste konnte eine kognitiv entlastende Interaktion erzielt werden.

Das Ressourcenrisiko bezüglich externer zeitlicher Verzögerungen (R-03) erforderte im Projektverlauf eine differenzierte Steuerung. Zwischen dem formalen Projektstart (Konzeptionsphase) und der finalen Einreichung lag die Gesamtzeit (Brutto-Laufzeit) bei rund zehn bis elf Wochen. Dieser zeitliche Abstand entstand nicht durch Verzögerungen in der Entwicklung. Er resultierte vielmehr aus den regulären Feedback- und Begutachtungszyklen der Hochschule. Die effektive Netto-Bearbeitungszeit der Arbeitspakete entsprach mit etwa acht Wochen (zwei Wochen Konzeption, vier Wochen Architektur und Implementierung, zwei Wochen Testen und Finalisierung) dem in Tabelle 2 definierten Meilensteinplan. Durch die strikte Fokussierung auf das MVP und die Zurückstellung sekundärer Komfortfunktionen konnte die geplante Entwicklungszeit ohne Qualitätseinbußen eingehalten werden.

7 Fazit und Ausblick

Die vorliegende Portfolioarbeit dokumentiert den vollständigen, phasenorientierten Lebenszyklus des Softwaresystems Medical Tracker von der Problem- und Stakeholder-Analyse über die formale Spezifikation und MVC-Architektur bis hin zur empirischen Verifikation und Erfassung technischer Schulden. Ausgangspunkt des Entwicklungsvorhabens war die prekäre Situation im häuslichen Pflegealltag, in dem analoge Medienbrüche und unstrukturierte Notizen das Risiko von Medikationsfehlern erhöhen und die Kommunikation mit behandelnden Ärzten erschweren. Als ingenieurwissenschaftliche Leitlinie diente die in Abschnitt 1.1 formulierte Forschungsfrage:

„Wie muss eine lokale, leichtgewichtige Webanwendung architektonisch konzipiert sein, um pflegende Angehörige bei der Medikations- und Vitaldatenverwaltung zu entlasten? Wie lässt sich dabei eine barrierefreie Nutzung für pflegebedürftige Personen realisieren, ohne die rechtliche Komplexität eines zertifizierten Medizinprodukts zu erzeugen?“

Die softwaretechnische Beantwortung dieser Forschungsfrage gliedert sich in drei zentrale architektonische Dimensionen. Erstens erfordert die Entlastung pflegender Angehöriger eine Drei-Schichten-Architektur auf Basis von Python und Flask. Die Kombination aus Service-Layer-Muster und relationaler Persistenz (Flask-SQLAlchemy mit SQLite) sorgt dabei für eine strikte Modularisierung. Durch die Kapselung der Domänenlogik in Blueprints und die strikte Durchsetzung der Geschäftsregeln (GR-01 bis GR-04) wird die manuelle Fehleranfälligkeit minimiert. Der Kernprozess GP-03 synchronisiert die Einnahmequittierung atomar mit einer transaktionssicheren Lagerbestandsführung (FA-06, FA-07) unter voller Einhaltung der ACID-Kriterien. Sobald ein definierter Mindestbestand unterschritten wird, generiert das System automatisiert proaktive Nachbestellwarnungen auf dem Verwaltungsdashboard, wodurch Medikationsabrisse präventiv abgewendet werden. Gleichzeitig fangen typisierte Validierungsprüfungen unplausible Vitalwerte vor Erreichen der Persistenzschicht ab. ReportLab ermöglicht den In-Memory-Export strukturierter DIN-A4-Arztberichte (FA-03) in unter 1,3 Sekunden, was Konsultationen evidenzbasiert vorbereitet und Angehörige von aufwendigen handschriftlichen Zusammenfassungen befreit.

Zweitens gelingt die barrierefreie Einbindung pflegebedürftiger Personen durch ein konsequentes duales Benutzerschnittstellenkonzept. Anstatt eine universelle Benutzeroberfläche zu überfrachten, trennt das System das funktionsdichte Angehörigen-Dashboard physisch und ergonomisch von einer kognitiv stark reduzierten Tagesansicht (FA-05). Diese Single-Page-Ansicht fokussiert geriatrische Endanwender ausschließlich auf die Frage der unmittelbaren Tagesorientierung („Was muss ich jetzt einnehmen?“). Die messtechnische Evaluation belegt die Einhaltung der Barrierefreiheitsstandards nach WCAG 2.2. Die Benutzeroberfläche erfüllt das Konformitätsniveau WCAG AAA und kompensiert altersbedingte Seh- und Motorikdefizite zuverlässig. Maßgeblich hierfür sind eine Grundtypografie von mindestens 18 pt (24 px), Klickflächen von über 54 px Höhe sowie Kontrastverhältnisse von mindestens 7,8:1 (bis zu 14,2:1 bei Standardtexten). Das Post-Redirect-Get-Muster

(PRG) verhindert zudem versehentliche Doppelquittierungen bei unbedachten Browser-Aktualisierungen.

Drittens wird die rechtliche und regulatorische Komplexität eines zertifizierten Medizinprodukts nach der europäischen Medizinprodukte-Verordnung (MDR 2017/745) durch eine rigorose Positiv- und Negativabgrenzung des Systembereichs vermieden. Der Medical Tracker ist architektonisch als reines Dokumentations-, Bestands- und Orientierungsassistenzsystem im privaten Standalone-Betrieb konzipiert. Sämtliche diagnostischen Berechnungen, Wechselwirkungsanalysen oder therapeutischen Dosisempfehlungen sind explizit als Out-of-Scope definiert; die klinische Verantwortung verbleibt uneingeschränkt bei der behandelnden Ärzteschaft. Indem die Anwendung vollständig autark ohne externe Cloud-Infrastruktur oder Telemetrie auf dem lokalen Host-System operiert, greift für die Verarbeitung sensibler Gesundheitsdaten (Art. 9 Abs. 1 DSGVO) primär die europäische Haushaltsausnahme nach Art. 2 Abs. 2 lit. c DSGVO. Flüchtige In-Memory-Streams beim PDF-Export verhindern zudem persistente Dateispuren im Betriebssystem-Cache und verankern das Paradigma des Privacy by Design.

Die softwaretechnische Qualität der Realisierung spiegelt sich in den empirischen Testergebnissen wider: Eine Testabdeckung von 90 % Line Coverage über alle Systemkomponenten (über 91 % in Controllern und Formularen) sowie Antwortzeiten von unter 2 ms bei Datenbankabfragen und 33 ms beim Dashboard-Rendering unter einer Belastung von 10.000 historischen Datensätzen belegen eine robuste, performante und skalierungsfähige Codebasis.

Für künftige Entwicklungsiterationen eröffnet die offengelegte Architektur gezielte Erweiterungspfade, welche die dokumentierten technischen Schulden systematisch auflösen können: Auf Anwendungsebene empfiehlt sich die Kapselung der Blueprint-Routen durch ein rollenbasiertes Zugriffskontrollsystem, um die Trennung zwischen Patient und Angehörigem auch auf geteilten Familienrechnern softwareseitig abzusichern (TD-01). Die Kopplung verteilter Haushaltsgeräte (z. B. Tablet am Krankenbett und Desktop-PC des Angehörigen) lässt sich durch eine leichtgewichtige Server-Sent-Events-Route (SSE) synchronisieren, um Einnahmequittierungen ohne manuelle Seitenaktualisierung in Echtzeit zu spiegeln (TD-02). Zur Absicherung von Data-at-Rest gegen physischen Datendiebstahl bietet die Integration der SQLite-Treibererweiterung SQLCIPHER eine transparente 256-Bit-AES-Dateiverschlüsselung ohne Performanceverlust (TD-04). Sollte das System perspektivisch aus dem familiären Rahmen in den ambulanten Pflegedienstbetrieb überführt werden, bildet die ORM-Abstraktion über SQLAlchemy das Fundament für eine nahtlose Migration auf ein mandantenfähiges Client-Server-RDBMS wie PostgreSQL (TD-03, NFA-07). Im abgegrenzten Rahmen des privaten Pflegehaushalts hat der Medical Tracker den Nachweis erbracht, dass eine leichtgewichtige, barrierefreie und datenschutzkonforme Entlastung mit modernen Webtechnologien softwaretechnisch vollumfänglich realisierbar ist.

Literaturverzeichnis

Deutsches Institut für Normung. (2015). *Metrologische Aspekte nichtselbsttätiger Waagen* (DIN EN 45501:2015-04).

Deutsches Institut für Normung. (2019). *Medizinische elektrische Geräte — Teil 2-61: Besondere Festlegungen für die Sicherheit einschließlich der wesentlichen Leistungsmerkmale von Pulsoxi-metriegeräten* (DIN EN ISO 80601-2-61:2019-09).

Deutsches Institut für Normung. (2024). *Nichtinvasive Blutdruckmessgeräte - Teil 2: Klinische Prü-fung der intermittierenden automatisierten Bauart* (DIN EN ISO 81060-2:2024-08).

Glikson, M., Nielsen, J. C., Kronborg, M. B., Michowitz, Y., Auricchio, A., Barbash, I. M., Barrabés, J. A., Boriani, G., Braunschweig, F., Brignole, M., Burri, H., Coats, A. J. S., Deharo, J.-C., Delgado, V., Diller, G.-P., Israel, C. W., Keren, A., Knops, R. E., Kotecha, D., . . . Madhavan, M. (2021). 2021 ESC Guidelines on cardiac pacing and cardiac resynchronization therapy. *European Heart Journal*, 42(35), 3427–3520. <https://doi.org/10.1093/eurheartj/ehab364>

International Organization for Standardization. (2023). *Systems and software engineering — Sys-tems and software quality models* (ISO/IEC Standard No. 25010:2023).

McEvoy, J. W., McCarthy, C. P., Bruno, R. M., Brouwers, S., Canavan, M. D., Ceconi, C., Christo-dorescu, R. M., Daskalopoulou, S. S., Ferro, C. J., Gerdts, E., Hanssen, H., Harris, J., Lauder, L., McManus, R. J., Molloy, G. J., Rahimi, K., Regitz-Zagrosek, V., Rossi, G. P., Sandset, E. C., . . . Khamidullaeva, G. A. (2024). 2024 ESC Guidelines for the management of elevated blood pressure and hypertension. *European Heart Journal*, 45(38), 3912–4018. <https://doi.org/10.1093/eurhe-arti/ehae178>

Pohl, K. & Rupp, C. (2021). *Basiswissen Requirements Engineering: Aus- und Weiterbildung nach IREB-Standard zum Certified Professional for Requirements Engineering Foundation Level*.

W3C. (2023). *Web content accessibility guidelines (WCAG) 2.2* (W3C Recommendation). W3C.

Abstract

Die häusliche Pflege stellt für pflegende Angehörige eine immense organisatorische und emotionale Dauerbelastung dar. Unstrukturierte Papiernotizen, unübersichtliche Medikamentenschachteln und Medienbrüche begünstigen im Pflegealltag gravierende Fehler wie vergessene Einnahmen oder Fehldosierungen. Ziel des Entwicklungsvorhabens war die Realisierung des Medical Trackers als schlanke, lokale Webanwendung. Das System entlastet Angehörige bei der Medikations- und Vitaldatenverwaltung und bereitet Arztkonsultationen durch automatisierte PDF-Berichte fundiert vor. Gleichzeitig bietet eine barrierefreie Tagesansicht der pflegebedürftigen Person Orientierung und stärkt deren Selbstständigkeit.

Methodisch folgte das Vorhaben einem sequenziellen V-Modell, begleitet durch frühe UI-Mockups. In der Konzeptionsphase (Phase 1) wurden die Anforderungen entlang dreier Stakeholder-Gruppen in sieben User-Stories (FA-01 bis FA-07) operationalisiert, durch Qualitätsanforderungen präzisiert und in einer Requirements-Traceability-Matrix verankert. In der Erarbeitungsphase (Phase 2) erfolgte die softwarearchitektonische Umsetzung in einer Drei-Schichten-Architektur (MVC mit Service Layer). Auf Basis von Python und Flask wurden Datenmodelle via SQLAlchemy strukturiert, Geschäftsregeln (GR-01 bis GR-04) umgesetzt und der transaktionale Kernprozess der Medikamenteneinnahme (GP-03) über synchrone Kontrollflüsse realisiert. In der Finalisierungsphase (Phase 3) wurde die Anwendung einer formalen Qualitätssicherung unterzogen, durch automatisierte Lasttests verifiziert und im Hinblick auf verbliebene technische Schulden kritisch reflektiert.

Als zentraler Erfolgsfaktor erwies sich die strenge funktionale und ergonomische Trennung des Benutzerschnittstellenkonzepts. Anstelle eines überfrachteten Einheitslayouts trennt das System das informationsdichte Verwaltungsdashboard für Angehörige physisch von einer kognitiv stark reduzierten Tagesansicht für die zu pflegende Person. Die konsequente Orientierung an den Web Content Accessibility Guidelines (WCAG 2.2 AAA) mit Grundschriftgrößen von mindestens 18 pt (24 px), kontraststarker Farbgebung ($\geq 7:1$) und Buttons von über 54 px Höhe ermöglichte eine nachweislich barrierefreie Interaktion ohne visuelle Reizüberflutung.

Auf architektonischer Ebene bewährte sich das Zero-Configuration-Deployment: Durch die Wahl des leichtgewichtigen Frameworks Flask in Verbindung mit einer prozessintegrierten SQLite-Datendatenbank ist das System auf jedem modernen Standard-Windows-PC ohne Datenbankinstallation lauffähig. Zudem schützt die flüchtige In-Memory-Generierung strukturierter DIN-A4-Arztberichte via ReportLab sensible Gesundheitsdaten, indem 0 Byte temporäre Zwischendateien im Dateisystem verbleiben (Privacy by Design). Die Umsetzung der Traceability-Matrix gewährleistete dabei, dass jede Codekomponente direkt auf eine fachliche Anforderung zurückgeführt werden konnte.

Während der Implementierung traten zwei wesentliche architektonische Herausforderungen auf. Ursprünglich war die Einnahmedokumentation rein als Statusattribut innerhalb der Entität *Medikament* modelliert worden. Dies führte bei der Prozessanalyse jedoch zu dem Problem, dass historische

Einnahmen bei Dosisänderungen oder Löschungen überschrieben worden wären. Die Lösung bestand in einer formalen Refaktorisierung des Datenmodells: Durch die Einführung der eigenständigen Entität *EinnahmeProtokoll* wurde das statische Stammdatenobjekt erfolgreich von dynamischen Ausführungsereignissen entkoppelt, wodurch absolute Revisionssicherheit (GR-03) erreicht wurde.

Eine zweite Hürde lag im Schutz vor versehentlichen Mehrfachbuchungen, wenn Anwender im Webbrowser nach Bestätigung einer Einnahme die Seite aktualisierten. Dies wurde durch die strikte Implementierung des Post-Redirect-Get-Musters gelöst: Jeder POST-Request beantwortet das Backend mit einem HTTP 302 Redirect auf die GET-Route, wodurch doppelte Bestandsdekrementierungen technisch ausgeschlossen sind.

Hinsichtlich der Zeitplanung wurde der in Phase 1 definierte Acht-Wochen-Rahmen der effektiven Netto-Arbeitszeit (zwei Wochen Konzeption, vier Wochen Architektur und Implementierung, zwei Wochen Qualitätssicherung) präzise eingehalten. Die Differenz zur formalen Gesamtbruttozeit von rund zehn Wochen ergab sich planmäßig aus den tutoriellen Feedback- und Begutachtungsintervallen zwischen den Portfolioteilen, welche methodisch wertvoll für die iterative Verfeinerung genutzt wurden.

Trotz des erreichten Reifegrads offenbart die retrospektive Reflexion konkrete Verbesserungspotenziale: Um das System für Laienanwender installationsfrei zu halten, wurde im MVP bewusst auf ein komplexes Authentifizierungsmodul und ein rollenbasiertes Rechteschema verzichtet (TD-01). Wird die Anwendung auf einem geteilten Haushaltsrechner betrieben, ist der Zugriffsschutz daher primär organisatorischer Natur. Für künftige Ausbaustufen sollte eine leichtgewichtige Benutzer- und Rollentrennung via Flask-Login nachgerüstet werden. Zudem basiert die UI-Aktualisierung auf synchronem Server-Side-Rendering. Bei zeitgleicher Nutzung mehrerer Geräte (z. B. Tablet am Krankenbett und PC im Arbeitszimmer) wäre die Anbindung von Server-Sent Events zur verzögerungsfreien Echtzeit-Synchronisation des Dashboards sinnvoll (TD-02).

Die wichtigste Erkenntnis dieses Projekts liegt im hohen Mehrwert einer lückenlosen Vorab-Spezifikation: Die vor der Implementierung erstellten UML-Diagramme und die präzise Definition von Geschäftsregeln (GR-01 bis GR-04) verhinderten zeitraubende Code-Irrwege. Die automatisierte Testsuite bestätigte diese Entwurfsdisziplin mit 17 fehlerfreien Testläufen, einer Testabdeckung von 90 % sowie einer Skalierbarkeit bei über 10.000 Datensätzen. Für den privaten Pflegehaushalt beweist der Medical Tracker, dass moderne Webstandards eine leichtgewichtige und barrierefreie Entlastung ermöglichen. Durch den konsequenten Verzicht auf Cloud-Infrastrukturen bleibt die lokale Datenhoheit dabei vollumfänglich gewahrt. Gleichzeitig legt die offene Dokumentation der technischen Schulden dar, dass eine formale und kryptographische Härtung den zwingenden nächsten Entwicklungsschritt für einen über den familiären Schutzbereich hinausgehenden Realeinsatz darstellt.